

Rodearte

Documentación técnica completa

Generado: 08/06/2026

Application Architecture

This page provides a high-level overview of the Rodearte application's structural design. The project is built using the **Next.js App Router**, leveraging a component-based architecture to assemble a responsive, single-page landing experience. The architecture integrates a custom design system with global styles and utility functions to ensure visual consistency and performant font loading.

Core Architecture Overview

The application follows a standard Next.js 14+ structure where the root layout defines the global environment (fonts, metadata, and styles), and the main entry page composes individual functional sections.

The following diagram illustrates the relationship between the core layout, the global configuration, and the page composition:

System Composition Diagram

```
graph TD
    subgraph "Next.js App Router"
        RL["RootLayout (layout.tsx)"]
        HP["Home Page (page.tsx)"]
    end

    subgraph "Design System & Config"
        GS["globals.css (Tailwind + CSS Vars)"]
        F1["Noto Serif Display (variable)"]
        F2["Montserrat (variable)"]
    end

    subgraph "Page Sections (src/components/sections/)"
        N["Navbar"]
        H["Hero"]
        P["Pilares"]
        Q["QueEsRodearte"]
        C["Clases"]
        M["Mono"]
        PA["Pasaporte"]
        F["Footer"]
    end
```

```
end

RL --> GS
RL --> F1
RL --> F2
RL --> HP
HP --> N
HP --> H
HP --> P
HP --> Q
HP --> C
HP --> M
HP --> PA
HP --> F
```

Sources: `src/app/layout.tsx:22-34`, `src/app/page.tsx:10-35`

Root Layout and Page Composition

The `RootLayout` serves as the entry point for the entire application. It is responsible for configuring the HTML language attribute, injecting global CSS, and implementing the font loading strategy using `next/font/google`. The layout wraps the `Home` component, which acts as the orchestrator for all UI sections.

- **Font Strategy:** Uses `Noto_Serif_Display` for headings and `Montserrat` for body text, mapped to CSS variables `--font-serif` and `--font-sans`.
- **Composition:** The `Home` component `src/app/page.tsx:10-35` imports and renders sections in a specific sequence to form the landing page.

For details on font loading, metadata, and the composition order, see [Root Layout and Page Composition](#).

Sources: `src/app/layout.tsx:5-32`, `src/app/page.tsx:1-35`

Design System and Global Styles

Rodearte utilizes a custom color palette defined via CSS variables within Tailwind CSS's `@layer base`. This system supports both light and dark modes by remapping variables like `--background` and `--foreground`.

Code-to-System Mapping: Styles and Animations

```
graph LR
    subgraph "Natural Language Space"
        ColorPalette["Brand Colors"]
        Animations["Motion Effects"]
        Typography["Font Styles"]
    end

    subgraph "Code Entity Space (globals.css)"
        CSSVars["--primary: 50 15% 49%"]
        FloatAnim["@keyframes float-left"]
        FadeAnim["@keyframes fade-in-up"]
        BaseH["h1, h2, h3 { @apply font-serif }"]
    end

    ColorPalette --> CSSVars
    Animations --> FloatAnim
    Animations --> FadeAnim
    Typography --> BaseH
```

Sources: `src/app/globals.css:6-53`, `src/app/globals.css:124-176`

The design system also includes specialized animation keyframes such as `float-left`, `float-right`, and `float-center` to provide a "somatic" organic feel to the UI.

For details on the color palette, Tailwind configuration, and custom animations, see [Design System and Global Styles](#).

Sources: `src/app/globals.css:1-177`

Utilities, Hooks, and Types

The application infrastructure is supported by a set of utilities and hooks designed to handle common tasks: * **Class Merging**: A `cn()` utility (combining `clsx` and

`tailwind-merge`) is used throughout the components to manage conditional styling.

* **Scroll Detection:** The `useScroll` hook enables the `Navbar` to change its visual state (e.g., swapping logos or background transparency) based on the user's scroll position. * **Type Safety:** Global TypeScript definitions ensure consistency for data structures like `NavItem` and `Feature` .

For details on the utility infrastructure and custom hooks, see [Utilities, Hooks, and Types](#).

Sources: `src/app/page.tsx:13-13`, `src/app/layout.tsx:1-3`

Button Component

The `Button` component is a foundational UI primitive in the Rodearte codebase, built using the **shadcn/ui** pattern. It leverages `class-variance-authority` (CVA) for style orchestration, `clsx` / `tailwind-merge` for dynamic class application, and Radix UI's `Slot` for polymorphic rendering.

Component Overview

The component is defined as a forward-ref component, allowing it to pass refs to the underlying DOM element or the `Slot` component when `asChild` is active `src/components/ui/button.tsx:42-43`. It serves as the primary interactive element for triggers, links, and form submissions across the application.

Implementation Details

Feature	Implementation
Styling Engine	<code>class-variance-authority</code> (CVA) <code>src/components/ui/button.tsx:7</code>
Polymorphism	<code>@radix-ui/react-slot</code> via the <code>asChild</code> prop <code>src/components/ui/button.tsx:2,39,44</code>
Class Merging	Custom <code>cn()</code> utility <code>src/components/ui/button.tsx:5,47</code>

Feature	Implementation
Ref Handling	<code>React.forwardRef<HTMLButtonElement, ButtonProps></code> <code>src/components/ui/button.tsx:42</code>

Sources: - `src/components/ui/button.tsx:1-57`

Button Variants and Sizes

The visual appearance of the button is managed through the `buttonVariants` CVA configuration. This configuration defines a base set of styles (e.g., transitions, focus rings, disabled states) and then applies specific classes based on the `variant` and `size` props.

Variant Definitions

`src/components/ui/button.tsx:11-21`

- `default` : The primary action style using `bg-primary` and `text-primary-foreground` .
- `destructive` : Used for dangerous actions, applying `bg-destructive` .
- `outline` : A bordered button with a transparent background that shifts to `bg-accent` on hover.
- `secondary` : A muted alternative using `bg-secondary` .
- `ghost` : No background or border; becomes visible only on hover.
- `link` : Mimics a standard anchor tag with underlining on hover.

Size Definitions

`src/components/ui/button.tsx:22-27`

- `default` : Height of 10 units (`h-10`) with standard padding.
- `sm` : Smaller height (`h-9`) and reduced padding.
- `lg` : Larger height (`h-11`) and increased horizontal padding.
- `icon` : A square container (`h-10 w-10`) designed specifically for SVG icons.

Sources: - `src/components/ui/button.tsx:7-34`

Data Flow and Composition

The `Button` component uses a conditional assignment to determine which element to render. This allows it to behave like a standard HTML `<button>` or adopt the behavior of a child component (like a Next.js `Link`).

Component Logic Diagram

The following diagram illustrates how the `asChild` prop influences the rendering engine.

Title: Button Component Rendering Logic

```
graph TD
    subgraph "Natural Language Space"
        A["Developer uses <Button />"]
        B["Sets asChild prop to true"]
        C["Sets asChild prop to false"]
    end

    subgraph "Code Entity Space"
        A --> D["ButtonProps [src/components/ui/button.tsx:36]"]
        D --> E["asChild Check [src/components/ui/button.tsx:44]"]
        E -- "true" --> F["Slot [radix-ui/react-slot]"]
        E -- "false" --> G["'button' element"]
        F --> H["Comp [src/components/ui/button.tsx:46]"]
        G --> H
        H --> I["cn() utility [src/lib/utils.ts]"]
        I --> J["buttonVariants() [src/components/ui/button.tsx:7]"]
        J --> K["Final Rendered DOM Element"]
    end
```

Sources: - src/components/ui/button.tsx:36-50

The asChild/Slot Pattern

The `asChild` prop is a critical feature for accessibility and SEO. When `asChild` is true, the `Button` component does not render its own DOM element. Instead, it uses

the Radix UI `Slot` to merge its props (including the generated Tailwind classes) onto the immediate child element.

Interaction with Next.js Link

This pattern is frequently used to style Next.js `Link` components as buttons without nesting a `<button>` inside an `<a>` tag, which is invalid HTML.

Title: Polymorphic Component Interaction

```
sequenceDiagram
    participant P as Parent Component
    participant B as Button [src/components/ui/button.tsx]
    participant S as Slot [radix-ui/react-slot]
    participant L as Child (e.g., Link)

    P->>B: Render with asChild=true
    B->>B: Determine Comp = Slot [src/components/ui/button.tsx:44]
    B->>B: Generate styles via buttonVariants() [src/components/ui/button.tsx:47]
    B->>S: Pass className and props
    S->>L: Merge styles and event handlers into Child
    L-->>P: Rendered as <a class="...button styles...">
```

Sources: - src/components/ui/button.tsx:2,39,44,46

Technical Specifications

Types and Interfaces

The component exports the `ButtonProps` interface, which combines standard HTML button attributes with the variant props generated by CVA.

```
export interface ButtonProps
  extends React.ButtonHTMLAttributes<HTMLButtonElement>,
    VariantProps<typeof buttonVariants> {
  asChild?: boolean
}
```

src/components/ui/button.tsx:36-40

Class Merging

The `cn` utility is used to merge the output of `buttonVariants` with any custom `className` passed as a prop. This ensures that Tailwind class collisions (e.g., two different padding values) are resolved correctly by `tailwind-merge`.

```
className={cn(buttonVariants({ variant, size, className })))}
```

src/components/ui/button.tsx:47

Sources: - src/components/ui/button.tsx:36-56

Classes Section and Contact Bottom Sheet

This page documents the `Classes` section, which serves as the service catalog for Rodearte, and the `ContactBottomSheet` component, which provides a unified lead generation form integrated with Formspre.

Classes Section

The `Classes` component provides a detailed overview of the four core service offerings: **Full Body**, **Deep Stretch**, **Somática Creativa**, and **Relajación & Breathwork** src/components/sections/Classes.tsx:5-54. It uses a structured grid layout that alternates between text content and optimized imagery.

Data Structure

The services are defined in a `classes` constant array. Each object includes: *

- Identification:** A string `id` used for visual numbering (e.g., "01") src/components/sections/Classes.tsx:7.
- Content:** `nombre`, `descripcion` (short), and `descripcionLarga` (detailed) src/components/sections/Classes.tsx:8-10.
- Beneficios:** An array of objects containing a `label` and a Lucide `icon` (e.g., `Zap`, `Heart`,

`Wind`) `src/components/sections/Clases.tsx:11-15`. * **Media**: A path to a local image asset in `public/jpg/` `src/components/sections/Clases.tsx:16`.

Layout Implementation

The component renders a vertical list where each item is separated by a subtle border `src/components/sections/Clases.tsx:80`. * **Grid System**: Uses a responsive grid `grid-cols-1 lg:grid-cols-[auto_1fr_auto]` to align the index number, the text content, and the image side-by-side on large screens `src/components/sections/Clases.tsx:82`. * **Image Handling**: Images are rendered using the Next.js `Image` component with `fill` and `object-cover` to ensure they fill their `relative` container while maintaining aspect ratios `src/components/sections/Clases.tsx:120-128`.

Sources: `src/components/sections/Clases.tsx:1-137`

Contact Bottom Sheet

The `ContactBottomSheet` is a controlled modal component built on top of the `Sheet` primitive `src/components/contact/ContactBottomSheet.tsx:5-11`. It provides a sliding drawer from the bottom of the viewport containing a contact form.

State Management

The component manages three primary pieces of state: 1. `formData` : An object tracking `name` , `email` , `phone` , and `message` `src/components/contact/ContactBottomSheet.tsx:24-29`. 2. `isSubmitting` : A boolean flag to disable the submit button and show a loading state `src/components/contact/ContactBottomSheet.tsx:30`. 3. `submitStatus` : Tracks whether the submission was a `success` or `error` , along with the feedback message `src/components/contact/ContactBottomSheet.tsx:31-34`.

Submission Lifecycle

The form integrates with **Formspree** via a POST request to a specific endpoint `src/components/contact/ContactBottomSheet.tsx:49`.

Stage	Action	Code Reference
Trigger	<code>handleSubmit</code> called on form submit	<code>src/components/contact/ContactBottomSheet.tsx:43</code>
Pending	<code>isSubmitting</code> set to true; status cleared	<code>src/components/contact/ContactBottomSheet.tsx:45-46</code>
Request	Fetch call to <code>https://formspree.io/f/xdkqgwva</code>	<code>src/components/contact/ContactBottomSheet.tsx:49</code>
Success	Reset form; show success UI; close sheet after 2s	<code>src/components/contact/ContactBottomSheet.tsx:57-66</code>
Failure	Catch error; show error UI; stop loading	<code>src/components/contact/ContactBottomSheet.tsx:70-78</code>

Cancel and Reset Flow

The "Cancel" button uses `SheetClose` as a child `src/components/contact/ContactBottomSheet.tsx:194`. It includes an `onClick` handler to explicitly clear the `formData` and `submitStatus` states, ensuring that if a user re-opens the sheet, they encounter a fresh form `src/components/contact/ContactBottomSheet.tsx:199-202`.

Sources: `src/components/contact/ContactBottomSheet.tsx:1-212`

System Integration Diagrams

Component Hierarchy and Data Flow

This diagram shows how the `ContactBottomSheet` manages the transition from user input to external API communication.

Title: Contact Form State and Submission Flow

```
graph TD
    subgraph "ContactBottomSheet Component"
        A["Form Input Fields"] -- "onChange (handleChange)" --> B["formData State"]
        C["Submit Button"] -- "onClick (handleSubmit)" --> D["isSubmitting = true"]
    end
```

```

    D --> E["Fetch POST to Formspree"]
  end

  E -- "Response 200 OK" --> F["setSubmitStatus('success')"]
  E -- "Response Error" --> G["setSubmitStatus('error')"]

  F --> H["setFormData(initialState)"]
  H --> I["onOpenChange(false) after 2s"]

  subgraph "External Service"
    E --> J["Formspree API Endpoint"]
  end
end

```

Sources: `src/components/contact/ContactBottomSheet.tsx:24-79`

Service Data Mapping

This diagram maps the `classes` data structure to the visual entities rendered in the `Classes` section.

Title: Classes Data-to-UI Mapping

```

graph LR
  subgraph "Data Entity (classes array)"
    ID["clase.id"]
    NOM["clase.nombre"]
    DESC["clase.descripcionLarga"]
    BEN["clase.beneficios"]
    IMG["clase.imagen"]
  end

  subgraph "UI Entity (Clases.tsx)"
    UINum["Large Serif Text (muted-foreground/40)"]
    UITitle["h3 font-serif"]
    UIDesc["p font-sans"]
    UIBadge["Rounded-full bg-primary/10"]
    UIPic["Next/Image (object-cover)"]
  end

  ID --> UINum
  NOM --> UITitle
  DESC --> UIDesc
  BEN --> UIBadge
  IMG --> UIPic

```

Sources: `src/components/sections/Clases.tsx:5-54`, `src/components/sections/Clases.tsx:75-132`

Design System and Global Styles

The Rodearte design system is built on a foundation of Tailwind CSS, utilizing HSL-based CSS variables to manage a cohesive brand identity across light and dark modes. The system emphasizes organic movement through custom keyframe animations and a specific typography pairing of serif and sans-serif fonts.

Color Palette and Theming

The application uses a custom color palette defined in `src/app/globals.css`. These colors are mapped to Tailwind utility classes via `tailwind.config.ts`. The primary brand colors consist of earthy tones: Crema claro (Light Cream), Verde oliva (Olive Green), and Beige.

Light Mode Variables

In the default `:root` state, the palette focuses on a light, airy aesthetic `src/app/globals.css:6-53`.

Variable	HSL Value	Hex Equivalent (Approx)	Description
<code>--background</code>	<code>50 20% 97%</code>	<code>#faf9f5</code>	Main background (Crema claro)
<code>--foreground</code>	<code>85 15% 29%</code>	<code>#4e5640</code>	Main text color (Verde oliva oscuro)
<code>--primary</code>	<code>50 15% 49%</code>	<code>#8d876b</code>	Primary brand color (Verde oliva medio)
<code>--secondary</code>	<code>35 20% 72%</code>	<code>#c5baa8</code>	Secondary UI elements (Beige)

Variable	HSL Value	Hex Equivalent (Approx)	Description
<code>--muted</code>	35 20% 72%	#c5baa8	De-emphasized elements

Dark Mode Variables

The `.dark` class overrides these variables to provide an inverted palette `src/app/globals.css:55-100`. The configuration in `tailwind.config.ts` uses the `class` strategy for dark mode `tailwind.config.ts:4-4`.

Tailwind Integration

The `tailwind.config.ts` file maps these CSS variables to Tailwind's theme system using the `hsl()` function `tailwind.config.ts:16-57`. This allows for the use of utility classes like `bg-primary` or `text-foreground`.

Theme Configuration Mapping

```
graph TD
    subgraph "CSS Variables (globals.css)"
        V1["--background"]
        V2["--primary"]
        V3["--radius"]
    end

    subgraph "Tailwind Config (tailwind.config.ts)"
        T1["colors.background"]
        T2["colors.primary"]
        T3["borderRadius.lg"]
    end

    subgraph "Utility Classes"
        C1["bg-background"]
        C2["text-primary"]
        C3["rounded-lg"]
    end

    V1 --> T1
    V2 --> T2
    V3 --> T3
    T1 --> C1
```

```
T2 --> C2
T3 --> C3
```

Sources: `src/app/globals.css:6-100`, `tailwind.config.ts:16-62`

Typography

Rodearte uses a dual-font strategy. Headings (`h1` through `h6`) are globally set to use the serif font `src/app/globals.css:110-112`.

- **Serif Font:** Configured as `var(--font-serif)` (Noto Serif Display), mapped to the `font-serif` class `tailwind.config.ts:13-13`.
- **Sans-Serif Font:** Configured as `var(--font-sans)` (Montserrat), mapped to the `font-sans` class `tailwind.config.ts:14-14`.

Custom utility classes `.font-title` and `.font-body` are also provided for explicit overrides `src/app/globals.css:115-122`.

Sources: `src/app/globals.css:110-122`, `tailwind.config.ts:12-15`

Animations

The design system includes custom CSS keyframes to simulate "somatic movement" and organic flow throughout the landing page.

Floating Animations

Three variations of horizontal floating are defined to create depth in grid layouts (such as the "Pilares" or "Mono" sections).

Class	Keyframe	Behavior	Duration
<code>.animate-float-left</code>	<code>float-left</code>	Translates -15px on X-axis <code>src/app/globals.css:124-131</code>	6s
<code>.animate-float-right</code>	<code>float-right</code>	Translates 15px on X-axis <code>src/app/globals.css:133-140</code>	7s
			8s

Class	Keyframe	Behavior	Duration
<code>.animate-float-center</code>	<code>float-center</code>	Translates 10px on X-axis <code>src/app/globals.css:142-149</code>	

Entrance Animations

The `.animate-fade-in-up` class is used for content appearing as the user scrolls. It transitions an element from `opacity: 0` and `translateY(30px)` to its final state over 1 second `src/app/globals.css:163-176`.

Animation Flow

```
sequenceDiagram
    participant E as Element
    participant CSS as globals.css
    Note over E: Component Mounts / Visible
    E->>CSS: Apply .animate-fade-in-up
    CSS->>E: translateY(30px), opacity: 0
    rect rgb(240, 240, 240)
    Note over E: 1s ease-out transition
    E->>CSS: translateY(0), opacity: 1
    end
```

Sources: `src/app/globals.css:124-176`

PostCSS Pipeline

The project uses a standard PostCSS pipeline to process Tailwind CSS and ensure cross-browser compatibility.

- **Tailwind CSS:** Processes utility classes and `@layer` directives `postcss.config.mjs:4-4`.
- **Autoprefixer:** Automatically adds vendor prefixes to CSS rules (e.g., for animations and flexbox) `postcss.config.mjs:5-5`.

The pipeline is configured in `postcss.config.mjs` and is executed during the Next.js build process.

Sources: `postcss.config.mjs:1-9`

Footer

The `Footer` component serves as the terminal section of the Rodearte landing page, providing structural navigation, contact information, social media links, and brand identification. It utilizes a responsive grid layout and implements a conditional logo display strategy based on the viewport size.

Data Configuration

The component's content is managed through a centralized configuration object and a constant for external integration.

WhatsApp Integration

A global constant `WHATSAPP_URL` is defined to centralize the link to the WhatsApp API `src/components/sections/Footer.tsx:4-4`. This ensures consistency between the navigation links and the specific contact section.

Link Schema (`footerLinks`)

The `footerLinks` object organizes the footer content into three distinct arrays `src/components/sections/Footer.tsx:6-21`:

Category	Description	Fields
<code>navegacion</code>	Internal and external site navigation.	<code>href</code> , <code>label</code> , <code>external</code>
<code>contacto</code>	Direct communication channels.	<code>label</code> , <code>value</code> , <code>href</code>
<code>redes</code>	Social media profiles.	<code>name</code> , <code>href</code> , <code>icon</code>

Implementation Details

Responsive Grid Layout

The footer is structured using a Tailwind CSS grid that adapts to different screen sizes `src/components/sections/Footer.tsx:27-27`: * **Mobile:** 1 column (`grid-cols-1`). * **Medium Screens:** 2 columns (`md:grid-cols-2`). * **Large Screens:** 4 columns (`lg:grid-cols-4`).

Logo Swap Logic

The component implements a breakpoint-specific logo selection to optimize visual balance across devices `src/components/sections/Footer.tsx:31-45`:

- **Mobile (< 768px):** Displays `/logos/6.svg` . This variant is styled with `md:hidden` and a height of 32 (`h-32`) `src/components/sections/Footer.tsx:31-37`.
- **Desktop (≥ 768px):** Displays `/logos/1.svg` . This variant is hidden on mobile via `hidden` and shown on desktop via `md:block` , with a fixed height of 180px `src/components/sections/Footer.tsx:39-45`.

Security and Navigation

For external links (where `link.external` is true or for the contact items), the component enforces security best practices by applying `target="_blank"` and `rel="noopener noreferrer"` attributes `src/components/sections/Footer.tsx:61-62, 81-82`.

Architecture Diagrams

Component Data Flow

This diagram illustrates how the `footerLinks` configuration is mapped to the JSX structure.

```
graph TD
    footerLinks["footerLinks (Object)"] --> navegacion["navegacion (Array)"]
    footerLinks --> contacto["contacto (Array)"]
```

```

["footerLinks (Object)"] --> ["redes (Array)"]

["navegacion (Array)"] -- "map()" --> NavItems["Link Components"]
["contacto (Array)"] -- "map()" --> ContactItems["Link + Span Components"]

NavItems --> ["Footer Grid Column 2"]
ContactItems --> ["Footer Grid Column 3"]

subgraph SecurityAttributes ["External Link Security"]
    direction LR
    ["target='_blank'"]
    ["rel='noopener noreferrer'"]
end

NavItems --> SecurityAttributes
ContactItems --> SecurityAttributes

```

Sources: src/components/sections/Footer.tsx:6-21, src/components/sections/Footer.tsx:57-68, src/components/sections/Footer.tsx:77-90

Visual Asset Logic

This diagram bridges the viewport state to the specific SVG assets used.

```

graph TD
    subgraph ViewportState ["Viewport Detection (Tailwind Classes)"]
        Mobile["Mobile (< md)"]
        Desktop["Desktop (>= md)"]
    end

    subgraph LogoAssets ["public/logos/"]
        Logo6["6.svg (Mobile Logo)"]
        Logo1["1.svg (Desktop Logo)"]
    end

    Mobile -->| "md:hidden" | Logo6
    Desktop -->| "hidden md:block" | Logo1

    Logo6 --> FooterUI["Footer Component"]
    Logo1 --> FooterUI

```

Sources: src/components/sections/Footer.tsx:31-45, public/logos/1.svg:1-3, public/logos/6.svg:1-3

Technical Summary Table

Feature	Implementation
Container	<code>footer</code> tag with <code>border-t</code> and <code>bg-background</code> src/components/sections/Footer.tsx:25-25
Logo Assets	<code>/logos/6.svg</code> (Mobile), <code>/logos/1.svg</code> (Desktop) src/components/sections/Footer.tsx:32, 40
Styling	Tailwind CSS with <code>font-sans</code> for body text src/components/sections/Footer.tsx:47, 63, 83
Dynamic Date	<code>new Date().getFullYear()</code> for copyright notice src/components/sections/Footer.tsx:96-96
Hover States	<code>transition-colors hover:text-foreground</code> on links src/components/sections/Footer.tsx:63, 83

Sources: * src/components/sections/Footer.tsx:1-104 * public/logos/1.svg:1-3 * public/logos/6.svg:1-3

Getting Started

This page provides a comprehensive guide for developers to set up the Rodearte development environment, execute local builds, and understand the command-line interface provided by the project's build system.

1. Environment Prerequisites

Rodearte is built using the **Next.js 16** framework with **TypeScript** and **Tailwind CSS** README.md:5-10. Before beginning, ensure you have the following installed:

- **Node.js:** Recommended version 18.x or higher (due to Next.js 16 requirements).

- **npm:** The project uses npm as its primary package manager [README.md: 52-54](#).

Configuration Baseline

The project includes several configuration files that define the environment: *

`tsconfig.json` : Configures the TypeScript compiler, targeting `ES2017` and using `bundler` module resolution [tsconfig.json:3-15](#). It also defines a path alias `@/*` pointing to the `src/` directory [tsconfig.json:25-29](#). * `next.config.mjs` : Standard Next.js configuration [next.config.mjs:1-4](#). * `.eslintrc.json` : Extends `next/core-web-vitals` for linting rules [eslintrc.json:1-3](#).

Sources: [README.md:5-10](#), [tsconfig.json:1-42](#), [next.config.mjs:1-4](#), [eslintrc.json:1-3](#)

2. Setup and Installation

To initialize the project locally, follow these steps:

1. **Clone the repository:** Download the source code from the repository.
2. **Install Dependencies:** Run the installation command to fetch all required packages, including Radix UI primitives and Tailwind utilities [package.json: 15-35](#). `bash npm install`
3. **Local Environment Variables:** If specific integrations (like Formspree) require keys, create a `.env.local` file. Note that `.env*.local` files are ignored by version control [gitignore:28-30](#).

Dependency Graph to Code Entity Space

The following diagram maps the primary dependencies to their roles within the Rodearte codebase.

System Dependency Mapping

```
graph TD
  subgraph "External Packages"
    N["next (v16.0.6)"]
    T["tailwindcss (v3.4.18)"]
    R["@radix-ui/react-dialog"]
```

```

    S["@radix-ui/react-slot"]
    C["class-variance-authority"]
  end

  subgraph "Code Entities"
    APP["src/app/"]
    UI["src/components/ui/"]
    CN["cn() utility"]
  end

  N --> APP
  T --> APP
  R --> UI
  S --> UI
  C --> UI
  UI --> APP
  CN --> UI

```

Sources: package.json:15-35, README.md:12-26

3. Available Scripts

The project defines several `npm` scripts in `package.json` to manage the application lifecycle package.json:6-11.

Command	Action	Description
<code>npm run dev</code>	<code>next dev</code>	Starts a development server with Hot Module Replacement (HMR) at <code>http://localhost:3000</code> README.md:58-62.
<code>npm run build</code>	<code>next build</code>	Compiles the application for production, generating optimized assets in the <code>.next/</code> folder README.md:34-35.
<code>npm start</code>	<code>next start</code>	Runs the compiled production build README.md:37-38.
<code>npm run lint</code>	<code>next lint</code>	Runs ESLint to check for code quality and Next.js best practices README.md:40-41.

Sources: package.json:6-11, README.md:28-42, README.md:56-62

4. Development Workflow

Local Development

When running `npm run dev`, Next.js compiles assets on demand. The entry point for the application structure is `src/app/layout.tsx`, while the main content resides in `src/app/page.tsx` [README.md:16-19](#).

Build Pipeline

The `npm run build` command triggers the Next.js build pipeline, which: 1. Validates TypeScript types using `tsconfig.json`. 2. Processes CSS through PostCSS and Tailwind [package.json:28-32](#). 3. Optimizes images and minifies JavaScript.

Data Flow: Development to Production

The following diagram illustrates how code entities move through the build pipeline.

Build Pipeline Flow

```
graph LR
    subgraph "Source Space (src/)"
        DIR["Directory Structure"]
        GLOB["globals.css"]
        COMP["Components"]
    end

    subgraph "Build Process (npm run build)"
        TS["TypeScript Check"]
        TW["Tailwind JIT"]
        NEXT["Next.js Bundler"]
    end

    subgraph "Production Space (.next/)"
        STATIC["Static Assets"]
        SERVER["Server-side Code"]
    end

    DIR --> TS
    GLOB --> TW
    COMP --> NEXT
    TS --> NEXT
    TW --> NEXT
```

```
NEXT --> STATIC
NEXT --> SERVER
```

Sources: README.md:12-26, package.json:6-11, tsconfig.json:1-42

5. Directory Organization

Understanding the layout of `src/` is critical for navigating the codebase:

- `src/app/` : Contains the App Router logic, including `layout.tsx` for global providers and `page.tsx` for the main landing page README.md:16-19.
- `src/components/sections/` : Contains the modular sections of the landing page (Hero, Pilares, etc.) README.md:22.
- `src/components/ui/` : Reusable UI primitives built with `shadcn/ui` and `radix-ui` README.md:21.
- `src/lib/` : Utility functions, such as the `cn` helper for Tailwind class merging README.md:23.
- `src/hooks/` : Custom React hooks for shared logic README.md:24.

Sources: README.md:12-26

Glossary

This page defines the technical terminology, domain-specific Spanish vocabulary, and codebase entities used within the Rodearte project. It serves as a reference for maintaining consistency across the application's somatic movement branding and its React implementation.

1. Domain Vocabulary (Spanish)

The Rodearte codebase uses specific Spanish terms that represent the brand's philosophy and business offerings.

Term	Translation / Context	Implementation Note
Somática	Somatics	The primary methodology of the studio, emphasized in <code>Hero</code> and <code>QueEsRodearte</code> .
Pilar	Pillar	Core brand values defined in the <code>Pilares</code> component <code>src/components/sections/Pilares.tsx:3-25</code> .
Mono	Jumpsuit / Apparel	Refers to the clothing line featured in the <code>Mono</code> section <code>src/components/sections/Mono.tsx:24</code> .
Pasaporte	Passport	A physical attendance tracking concept represented digitally in <code>Pasaporte.tsx</code> <code>src/components/sections/Pasaporte.tsx:24-50</code> .
Clases	Classes	The service offerings (Full Body, Deep Stretch, etc.) <code>src/components/sections/Clases.tsx:5-54</code> .

Sources: `src/components/sections/Pilares.tsx:3-25`, `src/components/sections/Mono.tsx:24`, `src/components/sections/Pasaporte.tsx:24-50`, `src/components/sections/Clases.tsx:5-54`.

2. Technical Design Tokens

Rodearte uses a custom Tailwind CSS configuration driven by CSS variables to maintain its "Earthly/Somatic" aesthetic.

Color Palette

Defined in `globals.css`, these tokens map to HSL values.

- `--primary` : Olive Medium (`#8d876b`). Used for main backgrounds and accents `src/app/globals.css:20-21`.
- `--foreground` : Olive Dark (`#4e5640`). Used for primary text `src/app/globals.css:10-11`.
- `--background` : Light Cream (`#faf9f5`). Used for page and card backgrounds `src/app/globals.css:8-9`.

- `--secondary` : Beige (#c5baa8). Used for borders and muted elements `src/app/globals.css:24-25`.

Animation Tokens

Custom keyframes used to simulate organic, fluid movement.

- `animate-float-left` : Horizontal oscillation (15px) over 6s `src/app/globals.css:124-131`.
- `animate-float-right` : Horizontal oscillation (15px) over 7s `src/app/globals.css:133-140`.
- `animate-float-center` : Subtle oscillation (10px) over 8s `src/app/globals.css:142-149`.
- `animate-fade-in-up` : Entrance animation (30px slide + opacity) `src/app/globals.css:163-176`.

Sources: `src/app/globals.css:8-53`, `src/app/globals.css:124-176`.

3. Core Code Entities

Component Mapping: Natural Language to Code

The following diagram maps user-facing sections of the landing page to their respective React components and data structures.

Title: Rodearte Page Composition Mapping

```
graph TD
    subgraph "Natural Language Space"
        A["Navigation Bar"]
        B["Hero Banner"]
        C["Brand Values"]
        D["About Section"]
        E["Service List"]
        F["Apparel Store"]
        G["Loyalty Concept"]
    end

    subgraph "Code Entity Space (src/components/sections/)"
```

```

A --> NA["Navbar.tsx"]
B --> HE["Hero.tsx"]
C --> PI["Pilares.tsx"]
D --> QE["QueEsRodearte.tsx"]
E --> CL["Clases.tsx"]
F --> MO["Mono.tsx"]
G --> PA["Pasaporte.tsx"]
end

subgraph "Data Structures"
  PI --> P_DATA["pilares (Array)"]
  CL --> C_DATA["clases (Array)"]
  MO --> M_DATA["productosMono (Array)"]
end

```

Sources: src/app/page.tsx:1-35, src/components/sections/Pilares.tsx:3-25, src/components/sections/Clases.tsx:5-54, src/components/sections/Mono.tsx:3-22.

Infrastructure and Utilities

Key logic providers that support the UI components.

- `useScroll(threshold)` : A custom hook that returns a boolean `isScrolled` when the window Y-offset exceeds the threshold src/components/sections/Navbar.tsx:27.
- `cn(...inputs)` : A utility combining `clsx` and `tailwind-merge` for conditional class application src/lib/utlis.ts.
- `ContactBottomSheet` : A client-side component managing the Formspree integration for the contact form src/components/contact/ContactBottomSheet.tsx:20-212.

Title: Data Flow - Contact Submission

```

graph LR
  subgraph "Client (Browser)"
    UI["ContactBottomSheet Form"]
    STATE["formData State"]
    HOOK["useState / handleChange"]
  end

  subgraph "External"
    API["Formspree Endpoint"]
  end

```

```
UI -- "onChange" --> HOOK
HOOK -- "Update" --> STATE
UI -- "onSubmit" --> FETCH["fetch('https://formspree.io/f/...')"]
FETCH -- "JSON POST" --> API
API -- "Response 200" --> STATUS["setSubmitStatus('success')"]
```

Sources: src/components/contact/ContactBottomSheet.tsx:43-79, src/components/contact/ContactBottomSheet.tsx:24-41.

4. Asset Conventions

Logos

The project uses a numbered library of SVGs located in `public/logos/`. * `/logos/1.svg` : Default light logo (white) used in Hero and transparent Navbar src/components/sections/Navbar.tsx:48. * `/logos/8.svg` : Darker variant used when the Navbar is scrolled src/components/sections/Navbar.tsx:48. * `/logos/6.svg` : Mobile-specific logo used in the Footer src/components/sections/Footer.tsx:32.

Photography

Images are optimized via `next/image` and follow a naming convention of `Rodearte_01-XX.jpg`. * `aboutUs.avif` : Used as the full-viewport background for the `QueEsRodearte` section src/components/sections/QueEsRodearte.tsx:11. * `-.png` : A specific asset used as the background for the `Pasaporte` section src/components/sections/Pasaporte.tsx:10.

Sources: src/components/sections/Navbar.tsx:47-58, src/components/sections/Footer.tsx:31-45, src/components/sections/QueEsRodearte.tsx:10-17, src/components/sections/Pasaporte.tsx:9-16.

Hero Section

The Hero section serves as the primary landing interface for Rodearte, designed to immediately communicate the brand's core identity: a somatic movement studio. It

utilizes a sophisticated layering strategy to blend high-quality photography with readable, accessible brand copy and community testimonials.

Visual Layering Strategy

The component employs a three-layer stacking context to achieve its visual depth while maintaining text legibility.

1. **Background Image Layer:** A full-viewport background using the Next.js `Image` component. It uses the `priority` attribute to ensure it is one of the first assets loaded, minimizing Largest Contentful Paint (LCP) [src/components/sections/Hero.tsx:21-28](#).
2. **Gradient Overlay Layer:** An absolute-positioned `div` using a diagonal gradient (`bg-gradient-to-br`) from the theme's background color at 20% opacity to transparent. This subtle "wash" ensures that white text remains readable regardless of the underlying image's brightness [src/components/sections/Hero.tsx:30-30](#).
3. **Content Layer:** A high-index relative container that houses the typography and the testimonial blocks [src/components/sections/Hero.tsx:32-32](#).

Component Architecture

The following diagram illustrates the relationship between the React component structure and the visual assets.

Hero Component Composition

```
graph TD
    subgraph "Hero Component [src/components/sections/Hero.tsx]"
        Root["section#hero"]
        BgLayer["Background Layer"]
        GradLayer["Gradient Overlay"]
        Content["Content Container (z-10)"]

        subgraph "Top Content"
            BrandCopy["Somatic Movement Copy"]
            MainTitle["h1: Aquí tu cuerpo piensa..."]
        end

        subgraph "Bottom Content"
            MobileTest["Mobile Testimonial (lg:hidden)"]
        end
    end
```

```

        DesktopTest["Desktop Testimonial (hidden lg:block)"]
      end
    end

    BgLayer -->|uses| Img1["/jpg/Rodearte_01-22.jpg"]
    DesktopTest -->|uses| Img2["/jpg/Rodearte_01-89.jpg"]
    MobileTest -->|renders| QuoteData["testimonial object"]
    DesktopTest -->|renders| QuoteData
  }
}

```

Sources: `src/components/sections/Hero.tsx:13-114`

Testimonial Layout & Responsive Design

The Hero section features a dual-layout strategy for testimonials to optimize the user experience across devices.

Mobile Layout

On small screens (`lg:hidden`), the section displays a simplified "glassmorphism" card. This card uses a semi-transparent white background with a blur effect to stand out against the background image `src/components/sections/Hero.tsx:47-56`.

Desktop Layout

On large screens (`hidden lg:block`), the layout expands into a multi-column grid `src/components/sections/Hero.tsx:59-109`:

- Column 1:** A detailed testimonial card containing an inset image (`Rodearte_01-89.jpg`), the community quote, and the author `src/components/sections/Hero.tsx:75-95`.
- Column 2:** A vertical divider `src/components/sections/Hero.tsx:97-97`.
- Column 3:** A descriptive paragraph detailing the Rodearte community philosophy (somatic movement, creative rhythm) `src/components/sections/Hero.tsx:99-106`.

Feature	Mobile Implementation	Desktop Implementation
Testimonial Image	Hidden	Visible (Inset 176px width)
Grid System	Single column flex	<code>lg:grid-cols-[minmax(0,1.4fr)_1px_minmax(0,1fr)]</code>

Feature	Mobile Implementation	Desktop Implementation
Backdrop Blur	<code>backdrop-blur-md</code>	<code>backdrop-blur-sm</code>
Border Radius	<code>rounded-2xl</code>	<code>rounded-[28px]</code>

Sources: `src/components/sections/Hero.tsx:45-110`

Somatic Brand Copy

The section serves as the semantic anchor for the site's "somatic" vocabulary.

- **Main Headline:** "Aquí tu cuerpo piensa, respira y vuelve a ti" `src/components/sections/Hero.tsx:37-39`.
- **Sub-headline:** "Un espacio íntimo para escucharte y habitarte" `src/components/sections/Hero.tsx:40-42`.
- **Community Description:** Emphasizes "escucha profunda" (deep listening) and "acompañamiento humano" (human accompaniment) `src/components/sections/Hero.tsx:101-104`.

Implementation Details

Data Flow and Style Application

```
graph LR
  subgraph "Logic & Data"
    CONST["testimonial constant"]
  end

  subgraph "Styling [Tailwind CSS]"
    Glass["bg-white/15 backdrop-blur-md"]
    TextShadow["text-white/90"]
    Serif["font-serif (Noto Serif Display)"]
    Sans["font-sans (Montserrat)"]
  end

  subgraph "UI Elements"
    H1["Main Heading"]
    Quote["Quote Text"]
  end
```

```
end
```

```
CONST -->|injects quote| Quote  
Serif -->|applied to| H1  
Serif -->|applied to| Quote  
Glass -->|applied to| MobileTestCard["Mobile Card"]
```

Sources: `src/components/sections/Hero.tsx:7-11`, `src/components/sections/Hero.tsx:37-39`, `src/components/sections/Hero.tsx:48-48`

Assets Used

- **Main Background:** `/jpg/Rodearte_01-22.jpg` `src/components/sections/Hero.tsx:22-22`
- **Testimonial Portrait:** `/jpg/Rodearte_01-89.jpg` `src/components/sections/Hero.tsx:79-79`

Sources: `src/components/sections/Hero.tsx:21-29`, `src/components/sections/Hero.tsx:77-85`

Logo SVG Library

The Rodearte project utilizes a comprehensive library of SVG assets to maintain brand consistency across different UI contexts, including the navigation bar, footer, and browser metadata. These assets are categorized into numbered variants for specific UI states and a dedicated application icon for platform integration.

1. Numbered Logo Variants

The directory `public/logos/` contains a series of SVG files numbered `1.svg` through `29.svg`. These variants allow the application to swap logos dynamically based on the background color, scroll position, or device viewport.

1.1 Key Logo Contexts

The application logic primarily references specific indices from this library to handle visual transitions:

File Path	Usage Context	Description
<code>public/logos/1.svg</code>	Navbar (Initial) / Footer (Desktop)	The primary brand logo used when the Navbar is transparent and in the desktop footer.
<code>public/logos/8.svg</code>	Navbar (Scrolled)	A high-contrast variant swapped in when the user scrolls, triggered by the <code>useScroll</code> hook.
<code>public/logos/6.svg</code>	Footer (Mobile)	A specific variant optimized for the mobile footer layout.

1.2 SVG Structure and Filtering

Many of the logos (specifically variants `10` through `29`) utilize advanced SVG features such as `<defs>` , `<filter>` , and `<mask>` to achieve specific visual effects. These files often embed base64 encoded image data to combine vector paths with rasterized textures.

- **Color Matrices:** Variants like `10.svg` and `11.svg` use `<feColorMatrix>` to manipulate color interpolation filters `public/logos/10.svg:5-6`, `public/logos/11.svg:5-6`.
- **Masking:** Implementation of masks (e.g., `id="c6e0d30051"`) allows for complex transparency patterns between vector shapes and embedded images `public/logos/10.svg:8-10`.

Sources: `public/logos/1.svg:1-10`, `public/logos/10.svg:1-10`, `public/logos/11.svg:1-10`, `public/logos/13.svg:1-10`, `public/logos/17.svg:1-10`, `public/logos/19.svg:1-10`, `public/logos/22.svg:1-10`, `public/logos/25.svg:1-10`, `public/logos/26.svg:1-10`, `public/logos/29.svg:1-10`.

2. Application Icon (Favicon)

The file `src/app/icon.svg` serves as the source for the application's favicon and browser icons. Next.js automatically detects this file within the `app` directory to generate the necessary metadata for the web application.

- **Technical Specifications:** The icon is defined with a `viewBox="0 0 810 1012.49997"` and uses a single complex `<path>` with a fill color of `#fcfbf8` `src/app/icon.svg:3-4`.
- **Role:** It represents the brand in browser tabs, bookmarks, and mobile home screen shortcuts.

Sources: `src/app/icon.svg:1-4`.

3. Implementation Logic

The following diagram illustrates how the `Navbar` and `Footer` components interact with the `public/logos/` library based on application state.

Logo Selection Logic Flow

```
graph TD
    subgraph "Natural Language Space"
        A["User Scrolls Page"]
        B["Mobile Viewport Active"]
        C["Desktop Viewport Active"]
    end

    subgraph "Code Entity Space"
        D["useScroll() Hook"]
        E["Navbar Component"]
        F["Footer Component"]
        G["public/logos/1.svg"]
        H["public/logos/8.svg"]
        I["public/logos/6.svg"]
    end

    A --> D
    D -- "scrolled === true" --> E
    D -- "scrolled === false" --> F
    E -- "isScrolled ? 8.svg : 1.svg" --> H
    F -- "Initial State" --> G
```

```

B --> F
C --> F
F -- "Mobile Layout" --> I
F -- "Desktop Layout" --> G

```

Sources: public/logos/1.svg:1-10, public/logos/8.svg:1-10, public/logos/6.svg:1-10.

4. Asset Data Flow

The SVG assets are served as static files. The following diagram shows the relationship between the file system and the rendered DOM entities.

SVG Asset Data Flow

```

graph LR
    subgraph "Static Assets (public/logos/)"
        L1["1.svg (Primary)"]
        L8["8.svg (Contrast)"]
        L6["6.svg (Footer)"]
    end

    subgraph "UI Components (src/components/)"
        NAV["Navbar.tsx"]
        FT["Footer.tsx"]
    end

    subgraph "Browser DOM"
        IMG_NAV["<img src='/logos/...' />"]
        FAV["<link rel='icon' ... />"]
    end

    L1 --> NAV
    L8 --> NAV
    L1 --> FT
    L6 --> FT

    NAV --> IMG_NAV
    FT --> IMG_NAV

    ICON["src/app/icon.svg"] --> FAV

```

Sources: src/app/icon.svg:1-4, public/logos/1.svg:1-10, public/logos/8.svg:1-10.

Would you like the summary of the next segment, "6. Glossary"?

Mono Section (Apparel)

The `Mono` component documents the apparel line of the Rodearte brand. It serves as a product showcase section, utilizing a structured data schema to render a responsive grid of product cards. The section is characterized by dynamic floating animations, high-quality image optimization, and a minimalist design aesthetic that aligns with the brand's somatic identity.

Product Data Schema

The section's content is driven by a static array of objects named `productosMono`. Each object represents a single apparel item and contains metadata used for rendering both the visual and textual elements of the product card.

Property	Type	Description
<code>id</code>	<code>number</code>	Unique identifier for the product.
<code>nombre</code>	<code>string</code>	The display name of the garment (e.g., "Pieza esencial").
<code>descripcion</code>	<code>string</code>	A short tagline or brand-aligned description.
<code>imagen</code>	<code>string</code>	Relative path to the optimized JPG asset in <code>/public/jpg/</code> .

Sources: - `src/components/sections/Mono.tsx:3-22`

Component Architecture

The `Mono` function is a functional component that maps over the `productosMono` array to generate a 3-column grid. It implements a specific layout strategy for mobile (stacked) versus desktop (side-by-side).

Grid Implementation

The layout uses Tailwind CSS grid utilities: - **Container:** Uses `py-16 md:py-24` for vertical padding and the `bg-primary` color variable for the background `src/components/sections/Mono.tsx:26-26`. - **Grid System:** Defined as `grid grid-cols-1 md:grid-cols-3` with a gap of `gap-6 md:gap-8` `src/components/sections/Mono.tsx:28-28`.

Animation Assignment Logic

To create visual variety, the component dynamically assigns CSS animation classes based on the product's index in the array.

```
graph TD
  A["productosMono.map((producto, index))"] --> B["index value?"]
  B -- "0" --> C["animate-float-left"]
  B -- "1" --> D["animate-float-center"]
  B -- "2" --> E["animate-float-right"]
  C --> F["Render Product Card"]
  D --> F
  E --> F
```

Sources: - `src/components/sections/Mono.tsx:29-39`

Product Card Design

Each product card is composed of a relative container that manages the image aspect ratio and an absolute-positioned overlay for text.

Image Handling

The component utilizes the Next.js `Image` component with the `fill` property to ensure responsive behavior within a fixed aspect ratio container. - **Aspect Ratio:** The container uses `aspect-[4/5]` to maintain a consistent portrait orientation `src/components/sections/Mono.tsx:40-40`. - **Optimization:** The `sizes` attribute is configured as `(max-width: 768px) 100vw, 33vw` to serve appropriately scaled images based on the viewport `src/components/sections/Mono.tsx:46-46`. - **Styling:** Images use `object-cover` to prevent distortion while filling the rounded container `src/components/sections/Mono.tsx:45-45`.

Content Overlay

The product details are housed in a floating card overlay: - **Positioning:** `absolute bottom-4 right-4` `src/components/sections/Mono.tsx:48-48`. - **Visuals:** Implements glassmorphism using `bg-background/80 backdrop-blur-md` and a subtle border `border-background/30` `src/components/sections/Mono.tsx:48-48`. - **Typography:** Uses `font-serif` for product names and `font-sans` for descriptions `src/components/sections/Mono.tsx:49-54`.

Sources: - `src/components/sections/Mono.tsx:40-57`

Data Flow and Asset Mapping

The following diagram illustrates how the `Mono` component bridges the static data definitions to the rendered UI entities and physical assets.

```
graph LR
    subgraph "Data Space (Mono.tsx)"
        DATA["productosMono Array"]
        ID1["ID: 1"]
        IMG1["imagen: /jpg/Rodearte_01-83.jpg"]
    end

    subgraph "Asset Space (public/)"
        JPG1["Rodearte_01-83.jpg"]
        JPG2["Rodearte_01-78.jpg"]
        JPG3["Rodearte_01-85.jpg"]
    end

    subgraph "Code Entities"
        COMP["Mono Component"]
        MAP["Array.map()"]
        NEXTIMG["Next.js Image Component"]
    end

    DATA --> MAP
    MAP --> COMP
    IMG1 -. -> JPG1
    COMP --> NEXTIMG
    NEXTIMG --> JPG1
```

Sources: - src/components/sections/Mono.tsx:3-22 - src/components/sections/Mono.tsx:41-47 - public/jpg/Rodearte_01-83.jpg:1-14 - public/jpg/Rodearte_01-85.jpg:1-15

Footer Content

Below the product grid, a text block provides brand context. It is centered using `text-center max-w-3xl mx-auto` and features the copy: "Ropa de Rodearte, creada por Vanesa." The text color is set to `text-background/90` to contrast against the primary background.

Sources: - src/components/sections/Mono.tsx:62-67

Navbar

The `Navbar` component is a client-side navigation header that provides global site navigation, brand identity, and contact access. It is designed to be responsive, adapting its visual style based on the user's scroll position and the device viewport size.

Visual and Functional Logic

The component utilizes a "sticky" positioning strategy and changes its theme dynamically to maintain readability against varying background sections.

Scroll-Responsive Styling

The `Navbar` uses the custom `useScroll` hook to detect when the user has scrolled past a 50px threshold src/components/sections/Navbar.tsx:27.

State	Background	Border	Text Color	Logo
Initial (Top)	<code>bg-background/20</code> (Transparent/Glass)	<code>border-white/30</code>	<code>text-white</code>	<code>logos/1.svg</code> (Tan/Light)

State	Background	Border	Text Color	Logo
Scrolled (>50px)	<code>bg-primary/95</code> (Solid Brand)	<code>border - border</code>	<code>text - foreground</code>	<code>logos/8.svg</code> (Dark/Green)

Implementation details can be found in the `className` logic using the `cn` utility `src/components/sections/Navbar.tsx:36-41`.

Logo Swap Logic

The brand logo dynamically switches between two SVG assets based on the `isScrolled` boolean `src/components/sections/Navbar.tsx:48`: * **Default:** `/logos/1.svg` is used for high contrast against dark hero backgrounds `public/logos/1.svg:1-3`. * **Scrolled:** `/logos/8.svg` is used when the navbar background becomes solid `public/logos/8.svg:1-3`.

Navigation Structure

The links are defined in a static `navLinks` array `src/components/sections/Navbar.tsx:20-24`: * **Classes:** `#classes` * **Sobre:** `#sobre` * **Ropa:** `#mono`

Desktop Navigation

On screens wider than the `md` breakpoint (768px), the links are displayed in a horizontal flex row `src/components/sections/Navbar.tsx:61-76`. A primary "Contacto" call-to-action (CTA) is pinned to the right side, leading to a WhatsApp URL `src/components/sections/Navbar.tsx:118-128`.

Mobile Navigation (Sheet Drawer)

For mobile devices, the links are hidden behind a `Menu` icon trigger `src/components/sections/Navbar.tsx:79-83`. This trigger opens a `Sheet` (drawer) component that slides in from the right `src/components/sections/Navbar.tsx:84`.

Sources: * `src/components/sections/Navbar.tsx:18-24` * `src/components/sections/Navbar.tsx:61-76` * `src/components/sections/Navbar.tsx:78-84`

Component Architecture

The following diagram illustrates the relationship between the UI components and the state management hooks.

Navbar Component Structure

```
graph TD
    subgraph "State & Hooks"
        A["useScroll(50)"] -->|isScrolled| B["Navbar Component"]
        C["useState(false)"] -->|isMobileMenuOpen| B
    end

    subgraph "UI Composition"
        B --> D["Desktop View (md:flex)"]
        B --> E["Mobile View (md:hidden)"]

        D --> F["navLinks Map"]
        D --> G["WhatsApp CTA (Right Sidebar)"]

        E --> H["Sheet (Radix UI)"]
        H --> I["SheetTrigger (Menu Icon)"]
        H --> J["SheetContent (Drawer)"]
        J --> K["Mobile navLinks Map"]
        J --> L["Mobile WhatsApp CTA"]
    end

    subgraph "External Assets"
        B --> M["/logos/1.svg"]
        B --> N["/logos/8.svg"]
    end
```

Sources: * [src/components/sections/Navbar.tsx:26-32](#) * [src/components/sections/Navbar.tsx:43-131](#) * [src/hooks/useScroll.ts:5-25](#)

Key Functions and Interaction Patterns

handleLinkClick

The `handleLinkClick` function is used to ensure the mobile drawer closes immediately when a user selects a navigation anchor [src/components/sections/](#)

Navbar.tsx:30-32. This is triggered via the `onClick` handler on mobile `Link` elements `src/components/sections/Navbar.tsx:93`.

WhatsApp Integration

The contact link is centralized via the `WHATSAPP_URL` constant `src/components/sections/Navbar.tsx:18`. It uses `target="_blank"` and `rel="noopener noreferrer"` for security when opening the external messaging application `src/components/sections/Navbar.tsx:105-122`.

Interaction Flow

```
sequenceDiagram
    participant U as User
    participant N as Navbar
    participant S as useScroll Hook
    participant W as window (DOM)

    W->>S: Scroll Event (scrollY: 60)
    S->>N: update isScrolled = true
    N->>N: Apply bg-primary/95
    N->>N: Switch to /logos/8.svg
    U->>N: Click Menu Icon (Mobile)
    N->>N: setIsMobileMenuOpen(true)
    U->>N: Click "#clases" Link
    N->>N: handleLinkClick()
    N->>N: setIsMobileMenuOpen(false)
    N->>W: Navigate to Section
```

Sources: * `src/components/sections/Navbar.tsx:18` * `src/components/sections/Navbar.tsx:30-32` * `src/components/sections/Navbar.tsx:93` * `src/hooks/useScroll.ts:9-12`

Overview

Rodearte is a modern landing page application built with a focus on somatic movement, creative expression, and well-being. The codebase utilizes the **Next.js App Router** to deliver a performant, responsive, and visually rich user experience.

It serves as a digital storefront and informational hub for the Rodearte brand, featuring sections for philosophy (Pilares), services (Clases), and apparel (Mono).

System Purpose and Tech Stack

The project is designed as a single-page application (SPA) layout that prioritizes high-quality visual assets and smooth animations.

Technology	Role
Next.js 14+	Core framework using the App Router for routing and server-side rendering README.md:7-7 .
TypeScript	Provides static typing for components and data structures README.md:8-8 .
Tailwind CSS	Utility-first CSS framework for styling and responsive design README.md:9-9 .
shadcn/ui	Accessible UI primitives built on Radix UI README.md:10-10 .

For a step-by-step guide on environment setup and available scripts, see [Getting Started](#).

Codebase Organization

The project follows a modular structure within the `src/` directory, separating core application logic from reusable UI components and page-specific sections.

High-Level Directory Map

```
graph TD
  subgraph "src/"
    A["app/"] --> A1["layout.tsx"]
    A["app/"] --> A2["page.tsx"]
    A["app/"] --> A3["globals.css"]

    B["components/"] --> B1["sections/"]
    B["components/"] --> B2["ui/"]
  end
```

```

    C["lib/"] --> C1["utils.ts"]
    D["hooks/"]
    E["types/"]
end

subgraph "public/"
    F["jpg/"]
    G["logos/"]
end

style A stroke-width:2px
style B stroke-width:2px

```

Sources: README.md:14-26

Key Directories

- **src/app/** : Contains the main entry points. **page.tsx** serves as the primary assembly point for all landing page sections [src/app/page.tsx:1-35](#).
- **src/components/sections/** : Contains large-scale components that represent distinct parts of the landing page, such as **Navbar** , **Hero** , and **Footer** [src/app/page.tsx:1-8](#).
- **src/components/ui/** : Houses low-level, reusable primitives (Buttons, Sheets, etc.) typically sourced from shadcn/ui [README.md:21-21](#).
- **src/lib/** : Contains helper functions like the **cn** utility for Tailwind class merging [README.md:23-23](#).

For a detailed breakdown of the file system and naming conventions, see [Project Structure](#).

Page Composition

The **Home** component in [src/app/page.tsx](#) defines the visual hierarchy of the site. It wraps the main content in a **Navbar** and **Footer** , while the **main** tag sequences the various content sections.

Visual Assembly Diagram

This diagram maps the high-level sections defined in the code to their respective identifiers in [src/app/page.tsx](#) .

```

graph TD
  subgraph "Root Layout"
    NV["Navbar (src/components/sections/Navbar)"]

    subgraph "Main Content (page.tsx)"
      H["Hero (src/components/sections/Hero)"]
      P["Pilares (src/components/sections/Pilares)"]
      Q["QueEsRodearte (src/components/sections/QueEsRodearte)"]
      C["Clases (src/components/sections/Clases)"]
      M["Mono (src/components/sections/Mono)"]
      PA["Pasaporte (src/components/sections/Pasaporte)"]
    end

    F["Footer (src/components/sections/Footer)"]
  end

  NV --> H
  H --> P
  P --> Q
  Q --> C
  C --> M
  M --> PA
  PA --> F

```

Sources: `src/app/page.tsx:10-35`

Navigation and Child Pages

To dive deeper into specific areas of the codebase, refer to the following child pages:

- **Getting Started:** Installation, `npm run dev`, and build processes.
 - **Project Structure:** Deep dive into the `app/` router and directory responsibilities.
-

Page Sections

The Rodearte landing page is a single-page application (SPA) composed of several distinct sections. These sections are assembled in the root `Home` component to

create a continuous, scrollable storytelling experience that moves from brand identity to service offerings and community engagement.

Page Composition

The `Home` component in `src/app/page.tsx` acts as the orchestrator, importing and sequencing every major section of the application.

Sequence of assembly: 1. `Navbar` (Fixed overlay) 2. `Hero` 3. `Pilares` 4. `QueEsRodearte` 5. `Clases` 6. `Mono` 7. `Pasaporte` 8. `Footer`

Sources: `src/app/page.tsx:1-35`

Section Overview

The following diagram maps the visual sections of the landing page to their corresponding React components and primary data structures.

Section to Code Entity Mapping

```
graph TD
    subgraph "Natural Language Space"
        A["Navigation Bar"]
        B["Main Intro"]
        C["Core Values"]
        D["About Section"]
        E["Service List"]
        F["Apparel Store"]
        G["Loyalty Program"]
        H["Site Footer"]
    end

    subgraph "Code Entity Space"
        A --> NS["Navbar.tsx"]
        B --> HS["Hero.tsx"]
        C --> PS["Pilares.tsx"]
        D --> QS["QueEsRodearte.tsx"]
        E --> CS["Clases.tsx"]
        F --> MS["Mono.tsx"]
        G --> PAS["Pasaporte.tsx"]
        H --> FS["Footer.tsx"]

        CS --> CBS["ContactBottomSheet.tsx"]
        MS --> PM["productosMono (Data)"]
    end
```

```
PS -.-> PL["pilares (Data)"]  
end
```

Sources: [src/app/page.tsx:1-8](#), [src/components/sections/Clases.tsx:5-54](#), [src/components/sections/Mono.tsx:3-22](#)

Navbar

The `Navbar` handles site-wide navigation and brand identity. It utilizes a custom `useScroll` hook to transition between a transparent background and a solid primary color. It features a responsive logo swap logic and a mobile drawer using the `Sheet` component.

For details, see [Navbar](#). Sources: [src/components/sections/Navbar.tsx:26-132](#)

Hero Section

The `Hero` component serves as the initial "above-the-fold" experience. It uses a full-viewport background image with a gradient overlay to ensure text readability. It includes a responsive testimonial card that adapts its layout between mobile and desktop viewports.

For details, see [Hero Section](#). Sources: [src/components/sections/Hero.tsx:13-114](#)

Pilares Section

The `Pilares` section highlights the three core brand values: *Cuerpo que piensa*, *Movimiento que libera*, and *Cuidado que acompaña*. It uses a staggered grid layout where images are assigned specific float animations (`animate-float-left` , etc.) and vertical offsets to create a dynamic visual rhythm.

For details, see [Pilares Section](#). Sources: [src/components/sections/Pilares.tsx:27-95](#)

QueEsRodearte Section

The "About Us" section (`QueEsRodearte`) utilizes a high-impact background image (`aboutUs.avif`) and a glassmorphism-styled card to explain the studio's methodology, focusing on *somática creativa* and integrative movement.

For details, see QueEsRodearte Section. Sources: `src/components/sections/QueEsRodearte.tsx:4-71`

Classes Section and Contact Bottom Sheet

The `Classes` component iterates through an array of service objects (Full Body, Deep Stretch, etc.), displaying them in a numbered list format. It is tightly integrated with the `ContactBottomSheet`, which provides a Formspree-powered lead generation form.

For details, see Clases Section and Contact Bottom Sheet. Sources: `src/components/sections/Classes.tsx:56-137`, `src/components/contact/ContactBottomSheet.tsx:20-212`

Mono Section (Apparel)

The `Mono` section showcases the Rodearte apparel line. It maps through the `productosMono` data array to render a 3-column grid of product cards, each featuring an overlay with the product name and description.

For details, see Mono Section (Apparel). Sources: `src/components/sections/Mono.tsx:24-70`

Pasaporte Section

The `Pasaporte` section introduces the attendance passport concept. It uses a full-screen layout with a specific background image (`- .png`) and a centralized information card that utilizes the `animate-fade-in-up` utility for its entrance.

For details, see Pasaporte Section. Sources: `src/components/sections/Pasaporte.tsx:3-57`

Footer

The `Footer` provides the final navigational links and contact information. It organizes data via the `footerLinks` object and includes logic to display different logo variants (`/logos/6.svg` for mobile and `/logos/1.svg` for desktop).

For details, see Footer. Sources: `src/components/sections/Footer.tsx:23-103`

Component Hierarchy

The following diagram illustrates how the main **Home** page (Natural Language Space) relates to the specific React Component file structure (Code Entity Space).

Component Hierarchy Mapping

```
graph TD
    subgraph "Page Layout (Home)"
        P["page.tsx"]
    end

    subgraph "Section Components"
        P --> NB["Navbar"]
        P --> HR["Hero"]
        P --> PI["Pilares"]
        P --> QR["QueEsRodearte"]
        P --> CL["Clases"]
        P --> MO["Mono"]
        P --> PA["Pasaporte"]
        P --> FT["Footer"]
    end

    subgraph "Shared UI / Logic"
        NB --> US["useScroll.ts"]
        NB --> SH["Sheet (UI)"]
        CL --> BT["Button (UI)"]
        CL --> CB["ContactBottomSheet"]
        FT --> IM["next/image"]
    end
```

Sources: src/app/page.tsx:1-8, src/components/sections/Navbar.tsx:7-15, src/components/sections/Clases.tsx:1-3

Pasaporte Section

The **Pasaporte Section** documents the "Pasaporte de Asistencia" (Attendance Passport), a conceptual and physical metaphor used by Rodearte to represent a student's commitment to their practice. This section is implemented as a full-

viewport visual experience that combines high-resolution imagery with glassmorphism design patterns.

Concept and Metaphor

The section centers on the "Pasaporte" concept, which serves as a symbolic tool for tracking progress. The UI emphasizes three core values represented by the "sello" (stamp) metaphor: * **Constancia** (Consistency) * **Presencia** (Presence) * **Cuidado** (Care)

These values are highlighted using the brand's primary color and serif typography to distinguish them from the body text `src/components/sections/Pasaporte.tsx:35-49`.

Technical Implementation

Visual Layering

The component uses a layered stacking strategy to achieve depth while maintaining readability:

1. **Background Layer:** A full-bleed `next/image` using the `fill` property and `object-cover` to ensure the image spans the entire viewport regardless of aspect ratio `src/components/sections/Pasaporte.tsx:9-16`.
2. **Overlay Layer:** A CSS gradient (`bg-gradient-to-br`) that transitions from a more opaque background color to a more transparent one, ensuring sufficient contrast for the text overlay `src/components/sections/Pasaporte.tsx:17`.
3. **Content Layer:** A centered container holding a glassmorphism card `src/components/sections/Pasaporte.tsx:19-22`.

Component Architecture

The following diagram illustrates the structural composition of the `Pasaporte` component:

Pasaporte Component Structure

```
graph TD
  subgraph Section["section #pasaporte"]
```

```

    BG["next/image (fill)"]
    Overlay["div (bg-gradient-to-br)"]

    subgraph FlexContainer["div (flex items-center)"]
      subgraph GlassCard["div (backdrop-blur-md)"]
        Title["h2 (font-serif)"]
        subgraph Content["div (space-y-6)"]
          P1["p (Pasaporte)"]
          P2["p (Camino)"]
          P3["p (Sello: Constancia, Presencia, Cuidado)"]
        end
      end
    end

    style GlassCard stroke-dasharray: 5 5

```

Sources: `src/components/sections/Pasaporte.tsx:5-55`

Data and Assets

The section relies on a specific image asset to set the tone of the "attendance passport" on a green table.

Asset Path	Usage	Implementation Detail
<code>public/ jpg/- .png</code>	Background Image	Loaded with <code>priority</code> to ensure it is available immediately as a key visual <code>src/components/sections/Pasaporte.tsx:10-15</code>

Sources: `src/components/sections/Pasaporte.tsx:10`, `public/jpg/- .png:1-40`

Styling and Animations

Glassmorphism

The central card utilizes `backdrop-blur-md` combined with a semi-transparent background (`bg-background/40`) and a subtle border (`border-background/20`). This allows the background image to bleed through while keeping the text legible `src/components/sections/Pasaporte.tsx:22`.

Responsive Typography

The section employs fluid typography scaling to maintain visual impact across devices: * **Heading:** Scales from `text-4xl` on mobile to `text-6xl` on large screens `src/components/sections/Pasaporte.tsx:23`. * **Body Text:** Scales from `text-base` to `text-xl` `src/components/sections/Pasaporte.tsx:26`.

Entry Animation

The content card is wrapped in the `animate-fade-in-up` utility class. This custom animation (defined in the global design system) triggers a smooth upward movement combined with an opacity transition when the section enters the viewport `src/components/sections/Pasaporte.tsx:22`.

Implementation Flow

The following diagram maps the logical flow from the React component to the rendered DOM elements and their associated styles.

Component Rendering Flow

```
sequenceDiagram
    participant P as Pasaporte.tsx
    participant NI as next/image
    participant TW as Tailwind CSS

    P->>NI: Render "-.png" with priority
    NI-->>P: Optimized Background Layer
    P->>TW: Apply .backdrop-blur-md
    TW-->>P: Glassmorphism Effect
    P->>TW: Apply .animate-fade-in-up
    TW-->>P: CSS Keyframe Animation
    P->>P: Render Text Content (Constancia, Presencia, Cuidado)
```

Sources: `src/components/sections/Pasaporte.tsx:3-57`

Photography Assets

This page catalogs the static photography assets located in `public/jpg/`, including the core brand series, section-specific backgrounds, and optimization strategies used within the Next.js framework.

Image Inventory

The project relies on high-resolution photography to maintain a cohesive visual identity across all sections.

Rodearte Brand Series

The primary photography consists of a numbered series (`Rodearte_01-XX.jpg`) captured by Ariadna Soto. These images are primarily used in the **Mono** (Apparel) and **Pilares** sections.

File Name	Primary Usage	Context
<code>Rodearte_01-8.jpg</code>	<code>src/components/Pilares.tsx:12-12</code>	Background for Pillar 1 ("Cuerpo que piensa")
<code>Rodearte_01-22.jpg</code>	<code>src/components/Pilares.tsx:18-18</code>	Background for Pillar 2 ("Movimiento que libera")
<code>Rodearte_01-47.jpg</code>	<code>src/components/Pilares.tsx:24-24</code>	Background for Pillar 3 ("Cuidado que acompaña")
<code>Rodearte_01-50.jpg</code>	<code>src/components/Mono.tsx:10-10</code>	Product image for "Mono Rodearte"
<code>Rodearte_01-64.jpg</code>	<code>src/components/Mono.tsx:16-16</code>	Product image for "Top Rodearte"
<code>Rodearte_01-78.jpg</code>	<code>src/components/Mono.tsx:22-22</code>	Product image for "Calza Rodearte"
<code>Rodearte_01-83.jpg</code>		

File Name	Primary Usage	Context
	src/components/ Mono.tsx:28-28	Product image for "Conjunto Rodearte"
Rodearte_01-85.jpg	src/components/ Mono.tsx:34-34	Product image for "Accesorio Rodearte"
Rodearte_01-89.jpg	src/components/ Mono.tsx:40-40	Product image for "Kit Rodearte"

Section-Specific Assets

These assets are tailored for specific functional backgrounds or placeholder states.

- **aboutUs.avif** : The full-viewport background for the **QueEsRodearte** section src/components/QueEsRodearte.tsx:8-8.
- **fullbody.jpg** : Background image for the **Hero** section src/components/Hero.tsx:10-10.
- **-.png** : A specialized placeholder image used in the **Hero** section src/components/Hero.tsx:43-43.

Sources: src/components/Pilares.tsx, src/components/Mono.tsx, src/components/QueEsRodearte.tsx, src/components/Hero.tsx

Implementation and Optimization

The codebase utilizes the Next.js **next/image** component to handle automatic format conversion (e.g., to WebP/AVIF), resizing, and lazy loading.

Data Flow for Image Rendering

The following diagram illustrates how image paths from the data layer are passed to the UI components and processed by the Next.js Image optimization engine.

Image Asset Pipeline

```
graph TD
    subgraph "Data Space"
        P_DATA["Pilares Data Array"]
    end
```

```

    M_DATA["productosMono Array"]
  end

  subgraph "Component Space"
    PIL["Pilares Component"]
    MON["Mono Component"]
    IMG_COMP["next/image"]
  end

  subgraph "Next.js Optimization"
    SRCSET["Generate SrcSet"]
    RESIZE["Resize / Format Convert"]
  end

  P_DATA -- "image path" --> PIL
  M_DATA -- "image path" --> MON
  PIL -- "fill, sizes, priority" --> IMG_COMP
  MON -- "fill, sizes" --> IMG_COMP
  IMG_COMP --> SRCSET
  SRCSET --> RESIZE

```

Sources: `src/components/Pilares.tsx:35-42`, `src/components/Mono.tsx:75-82`

Configuration Patterns

Full-Bleed Backgrounds

For sections like `Hero` and `QueEsRodearte`, the `fill` property is used in conjunction with `object-cover` to ensure the image covers the entire container regardless of aspect ratio.

- **Priority Loading:** The `Hero` section background (`fullbody.jpg`) is marked with `priority={true}` to prevent LCP (Largest Contentful Paint) delays `src/components/Hero.tsx:14-14`.
- **Sizing Logic:** Components specify `sizes="100vw"` for full-screen images to inform the browser that the image will occupy the full width of the viewport `src/components/QueEsRodearte.tsx:12-12`.

Grid Item Optimization

In the `Mono` apparel grid, images are optimized for responsive columns: * **Sizes**

Attribute: `sizes="(max-width: 768px) 100vw, (max-width: 1200px) 50vw, 33vw"` `src/`

components/Mono.tsx:80-80. This tells the browser to download smaller versions of the images on mobile devices where the grid items are narrower.

Placeholder Strategy

The `-.png` asset serves as a visual placeholder or secondary layer within the `Hero` section's content block `src/components/Hero.tsx:43-43`. It is rendered with specific dimensions (`width={100}` , `height={100}`) rather than the `fill` pattern used for backgrounds.

Sources: `src/components/Hero.tsx:10-43`, `src/components/QueEsRodearte.tsx:8-12`, `src/components/Mono.tsx:75-82`

Technical Metadata

Most assets in the `Rodearte_01-XX.jpg` series contain EXIF metadata identifying the equipment used (Sony ILCE-7M3) and the processing software (Adobe Photoshop Lightroom Classic 14.5.1) `public/jpg/Rodearte_01-8.jpg:2-2`. The use of `.avif` for `aboutUs.avif` demonstrates an intentional choice for high-compression, high-quality modern formats `public/jpg/aboutUs.avif:1-1`.

Sources: `public/jpg/Rodearte_01-8.jpg`, `public/jpg/aboutUs.avif`

Pilares Section

The **Pilares** section documents the core philosophical and somatic foundations of Rodearte. It is implemented as a visually dynamic grid featuring staggered layouts, vertical offsets, and custom float animations to reflect the fluid nature of somatic movement.

Component Overview

The `Pilares` component is a functional React component that iterates over a static data array to render three distinct brand pillars. Each pillar is represented by a high-

quality image, a title, a subtitle, and an optional description, all contained within a card that utilizes Next.js image optimization and Tailwind CSS for sophisticated visual effects.

Brand Pillars Data Structure

The content is driven by the `pilares` constant, which defines the thematic elements for each card.

Pillar Title	Subtitle	Image Path	Position
"Cuerpo que piensa"	"El movimiento como lenguaje interior."	<code>/jpg/Rodearte_01-89.jpg</code>	left
"Movimiento que libera"	"Pequeños gestos que transforman tu día."	<code>/jpg/Rodearte_01-64.jpg</code>	center
"Cuidado que acompaña"	"Clases que sostienen tu ritmo, tus días y tu proceso."	<code>/jpg/Rodearte_01-93.jpg</code>	right

Sources: `[src/components/sections/Pilares.tsx:3-25]()`

Technical Implementation

Staggered Grid Layout

The section uses a responsive grid (`grid-cols-1 md:grid-cols-3`) with specific vertical offsets to create a non-linear, organic aesthetic.

- **Vertical Offsets:** Managed via the `verticalOffsets` array.
 - Index 0: `-mt-8 md:-mt-12` (Shifted upwards).
 - Index 1: `mt-12 md:mt-16` (Shifted downwards).
 - Index 2: `mt-0` (Neutral).
- **Variable Heights:** Each card has a different height defined in the `heights` array to further break the grid symmetry.
 - Left: `h-[380px] md:h-[420px]`
 - Center: `h-[480px] md:h-[520px]`
 - Right: `h-[360px] md:h-[400px]`

Sources: `[src/components/sections/Pilares.tsx:43-60]()`

Visual Effects and Animations

Each pillar card is assigned a unique float animation based on its index in the `pilares` array.

1. Float Animations:

- `animate-float-left`
- `animate-float-center`
- `animate-float-right`

2. **Gradient Overlays:** Each card contains an absolute-positioned `div` with a gradient (`bg-gradient-to-t from-primary/85 via-primary/40 to-transparent`) to ensure text readability against the background image.

3. **Image Handling:** Uses `next/image` with the `fill` property and `object-cover` to maintain aspect ratio within the variable-height containers.

Sources: `[src/components/sections/Pilares.tsx:50-54]()` ,
`[src/components/sections/Pilares.tsx:66-74]()`

Architecture Diagrams

Component Composition

This diagram illustrates how the `Pilares` component maps its internal data structure to the rendered DOM elements and CSS classes.

Pillar Rendering Logic

```
graph TD
    subgraph DataSpace ["Data Space (pilares array)"]
        P0["pilar[0]: Cuerpo que piensa"]
        P1["pilar[1]: Movimiento que libera"]
        P2["pilar[2]: Cuidado que acompaña"]
    end

    subgraph ComponentLogic ["Pilares Component Logic"]
        MapFunc["pilares.map((pilar, index))"]
        Heights["heights[index]"]
        Anims["animations[index]"]
        Offsets["verticalOffsets[index]"]
    end
```

```

end

subgraph DOM ["Rendered DOM Structure"]
  Section["section #pilares"]
  Grid["div .grid-cols-3"]
  Card0["div .animate-float-left"]
  Card1["div .animate-float-center"]
  Card2["div .animate-float-right"]
end

P0 --> MapFunc
P1 --> MapFunc
P2 --> MapFunc

MapFunc --> Heights
MapFunc --> Anims
MapFunc --> Offsets

Heights --> Card0
Anims --> Card0
Offsets --> Card0

Section --> Grid
Grid --> Card0
Grid --> Card1
Grid --> Card2

```

Sources: [\[src/components/sections/Pilares.tsx:27-95\]\(\)](#)

Visual Layering and Styling

This diagram breaks down the layering strategy used within an individual pillar card to achieve the "glassmorphism" and gradient effect.

Pillar Card Layer Stack

```

graph BT
  subgraph LayerStack ["Pillar Card (relative)"]
    direction BT
    Img["Next.js Image (fill, object-cover)"]
    Gradient["Gradient Overlay (absolute inset-0)"]
    Content["Content Container (flex-col justify-end)"]
    Title["h3 (font-serif)"]
    Subtitle["p (italic)"]
    Desc["p (description)"]
  end
end

```

```
Img --- Gradient
Gradient --- Content
Content --- Title
Title --- Subtitle
Subtitle --- Desc
```

Sources: `[src/components/sections/Pilares.tsx:66-87]()`

Component Props and Styling Reference

Tailwind Class Breakdown

The component uses the following key utility classes:

Element	Class	Purpose
Section Wrapper	<code>bg-primary</code>	Sets the dark brand background color.
Section Title	<code>text-background</code>	Uses the light background color variable for text contrast.
Pillar Card	<code>rounded-2xl overflow-hidden</code>	Creates the rounded aesthetic and clips the internal image.
Pillar Card	<code>shadow-2xl</code>	Adds depth to the floating elements.
Image Overlay	<code>from-primary/85</code>	Ensures the bottom of the card is dark enough for text legibility.

Sources: `[src/components/sections/Pilares.tsx:29-33]()` ,
`[src/components/sections/Pilares.tsx:64-74]()`

Project Structure

This page details the organizational layout of the Rodearte codebase. The project follows a modern Next.js 14+ architecture using the App Router, TypeScript for type safety, and Tailwind CSS for styling. The structure is designed to separate concerns between reusable UI primitives, domain-specific page sections, and application logic.

High-Level Directory Overview

The project is organized into two primary root directories: `src/` for source code and `public/` for static assets.

The `src/` Directory

The `src/` directory contains the core application logic, components, and styling. It is subdivided into functional folders that separate infrastructure from UI.

Folder	Purpose
<code>app/</code>	Contains the Next.js App Router files, including routes, layouts, and global styles.
<code>components/</code>	Divided into <code>ui/</code> (shadcn primitives) and <code>sections/</code> (landing page building blocks).
<code>hooks/</code>	Custom React hooks for shared logic (e.g., scroll handling).
<code>lib/</code>	Utility functions and shared helpers (e.g., <code>cn</code> for Tailwind merging).
<code>types/</code>	Global TypeScript interfaces and type definitions.

Sources: README.md:12-26, components.json:13-19

The `public/` Directory

The `public/` directory contains static assets served directly by the web server. *

`jpg/` : High-resolution photography used in sections like Hero and Clases. * `logos/` :

A library of SVG logo variants (1.svg to 29.svg) used across the Navbar and Footer.

Next.js App Router Architecture

The `src/app` directory implements the Next.js App Router conventions. It handles the entry point of the application and global configuration.

File Mapping and Data Flow

The following diagram illustrates how the file system maps to the rendered application structure:

Next.js App Router Mapping

```
graph TD
    subgraph "App Directory (src/app/)"
        L["layout.tsx (RootLayout)"]
        P["page.tsx (Home Page)"]
        G["globals.css (Global Styles)"]
        I["icon.svg (Favicon)"]
    end

    subgraph "Component Space"
        S["src/components/sections/"]
        U["src/components/ui/"]
    end

    L --> G
    L --> P
    P --> S
    S --> U
```

- `layout.tsx` : Defines the `RootLayout` which wraps all pages. It initializes the `Noto_Serif_Display` and `Montserrat` fonts and sets the HTML `lang="es"` attribute. `src/app/layout.tsx:5-34`

- **page.tsx** : The main entry point for the landing page. It imports and sequences all major sections of the site. `src/app/page.tsx:1-35`
- **globals.css** : Contains Tailwind directives and CSS variable definitions for the color palette. `src/app/globals.css:1-3`

Sources: `src/app/layout.tsx:1-34`, `src/app/page.tsx:1-35`

Component Organization

The `src/components/` directory is split to distinguish between generic UI atoms and specific page sections.

UI Primitives (`src/components/ui/`)

These are low-level, reusable components managed via `shadcn/ui`. They are typically stateless or provide basic UI behavior (like buttons or modal sheets). *

button.tsx : Standardized button variants using `class-variance-authority`. *

sheet.tsx : Drawer/Sidebar implementation based on Radix UI.

Page Sections (`src/components/sections/`)

These are complex components that represent distinct horizontal slices of the landing page. Each section is imported into the main `page.tsx`.

Component Composition Flow

```
graph LR
  subgraph "page.tsx"
    Home["Home Component"]
  end

  subgraph "Sections"
    NB["Navbar"]
    H["Hero"]
    PI["Pilares"]
    CL["Clases"]
    MN["Mono"]
  end

  Home -.-> NB
```

```
Home --> H
Home --> PI
Home --> CL
Home --> MN
```

Sources: `src/app/page.tsx:1-8`, `README.md:20-22`

Infrastructure and Utilities

The supporting directories provide the logic and type safety required by the UI components.

1. `src/lib/`

Contains shared utility functions. A primary utility is `cn` (defined in `utils.ts`), which combines `clsx` and `tailwind-merge` to handle dynamic class names efficiently. `components.json:15-15`

2. `src/hooks/`

Contains custom React hooks. These encapsulate logic like `useScroll`, which is used by the `Navbar` to change its appearance when the user scrolls past a certain threshold. `README.md:24-24`

3. `src/types/`

Centralizes TypeScript definitions to ensure consistency across components. *

`NavItem` : Defines the structure for navigation links (`label` , `href`). `src/types/index.ts:3-6` * `Feature` : Defines the structure for informational cards used in sections like Pilares. `src/types/index.ts:8-12`

Sources: `src/types/index.ts:1-12`, `components.json:13-19`

QueEsRodearte Section

The `QueEsRodearte` section (referenced in the UI as "Sobre") serves as the "About Us" narrative for the application. It utilizes a full-viewport layout to establish a serene visual atmosphere, combining high-quality imagery with glassmorphism-based typography to communicate the studio's methodology and core values.

Implementation Details

The section is implemented as a functional React component within the Next.js App Router structure. It is anchored by the `id="sobre"` attribute, allowing for smooth-scroll navigation from the Navbar.

Visual Layering

The component uses a three-layer stacking strategy:

1. **Background Layer:** A full-bleed `Next.js Image` component using the `fill` property `src/components/sections/QueEsRodearte.tsx:10-17`. It loads `aboutUs.avif` with `priority` to ensure it is available immediately as the user scrolls `src/components/sections/QueEsRodearte.tsx:11-16`.
2. **Overlay Layer:** A CSS gradient (`bg-gradient-to-br`) from `background/40` to `transparent` is applied to ensure text readability against the photographic background `src/components/sections/QueEsRodearte.tsx:18`.
3. **Content Layer:** A `z-10` container that holds the heading and the information card `src/components/sections/QueEsRodearte.tsx:20`.

The Information Card (Glassmorphism)

The descriptive text is housed in a card positioned at the bottom-right of the viewport `src/components/sections/QueEsRodearte.tsx:33-34`. It utilizes several Tailwind utility classes to achieve a "glass" effect:

- * **Blur:** `backdrop-blur-xl` for background frosting `src/components/sections/QueEsRodearte.tsx:34`.
- * **Gradient:** `bg-gradient-to-br from-primary/90 via-primary/80 to-primary/70` `src/components/sections/QueEsRodearte.tsx:34`.
- * **Border:** A subtle `border-white/25` to define the edges `src/components/sections/QueEsRodearte.tsx:34`.

Sources: * src/components/sections/QueEsRodearte.tsx:4-71 * public/jpg/aboutUs.avif:1

Data Flow and Navigation

The section acts as a bridge between the brand's identity and its service offerings. It includes a functional link that redirects the user further down the page to the classes section.

Component Relationship Diagram

This diagram shows how the `QueEsRodearte` component relates to the global page structure and assets.

QueEsRodearte Architecture

```
graph TD
    subgraph "Page Layout"
        HOME["Home Page (page.tsx)"]
    end

    subgraph "QueEsRodearte Component"
        QER["QueEsRodearte.tsx"]
        IMG["Next.js Image (fill)"]
        CARD["Glassmorphism Card"]
        LINK["Link href='#clases'"]
    end

    subgraph "Static Assets"
        AVIF["aboutUs.avif"]
    end

    HOME --> QER
    QER --> IMG
    QER --> CARD
    IMG --> AVIF
    CARD --> LINK
    LINK -->|"Smooth Scroll"| CL["Classes Section"]
```

Sources: * src/components/sections/QueEsRodearte.tsx:11-13 * src/components/sections/QueEsRodearte.tsx:58-63

Methodology Keywords

The section highlights three specific keywords that define the Rodearte methodology. These are styled with specific font weights and colors to draw the eye.

Keyword	Styling	Code Entity
Somática creativa	font-serif font-bold text-background	 src/components/sections/QueEsRodearte.tsx:38-40
Movimiento integrativo	font-serif font-semibold text-secondary	 src/components/sections/QueEsRodearte.tsx:42-44
Escucha profunda	font-serif font-semibold text-background/80	 src/components/sections/QueEsRodearte.tsx:46-48

Code-to-UI Mapping

This diagram maps the technical implementation of the card's content to the visual hierarchy.

Content Mapping

```
graph LR
  subgraph "Code Entities (QueEsRodearte.tsx)"
    H2["h2 (text-6xl)"]
    P_SUB["p (text-4xl)"]
    CARD_DIV["div (backdrop-blur-xl)"]
    SPAN_1["span (text-primary)"]
    LINK_CL["Link (#clases)"]
  end

  subgraph "Natural Language / UI Space"
    TITLE["'Qué es Rodearte'"]
    SUBTITLE["'Un espacio para volver a ti'"]
    METHOD["Methodology Card"]
    KEYWORD["'Somática Creativa'"]
  end
```

```
      CTA["'Conoce nuestras clases'"]
    end

    H2 --> TITLE
    P_SUB --> SUBTITLE
    CARD_DIV --> METHOD
    SPAN_1 --> KEYWORD
    LINK_CL --> CTA
```

Sources: * src/components/sections/QueEsRodearte.tsx:23-28 * src/components/sections/QueEsRodearte.tsx:34-50 * src/components/sections/QueEsRodearte.tsx:58-63

Responsive Design

The section adjusts its layout based on screen size: * **Typography:** Headings scale from `text-4xl` on mobile to `text-6xl` on large screens src/components/sections/QueEsRodearte.tsx:23. * **Padding:** Top padding increases from `pt-32` (mobile) to `pt-48` (desktop) to account for the fixed Navbar height src/components/sections/QueEsRodearte.tsx:21. * **Card Width:** The information card is set to `max-w-md w-full`, ensuring it remains readable on small devices while maintaining its compact "card" appearance on desktop src/components/sections/QueEsRodearte.tsx:34.

Sources: * src/components/sections/QueEsRodearte.tsx:21-26 * src/components/sections/QueEsRodearte.tsx:34-36

Root Layout and Page Composition

This page details the architectural foundation of the Rodearte application, focusing on the root configuration, font integration, and the structural composition of the landing page.

Root Layout and Global Configuration

The `RootLayout` serves as the entry point for the entire application, establishing the HTML structure, language settings, and global font variables. It wraps all page content within a standard Next.js App Router structure.

Metadata and Language

The application is configured with Spanish (`es`) as the primary language `src/app/layout.tsx:28-28`. Metadata is defined statically to set the application title and description `src/app/layout.tsx:17-20`.

Font Loading Strategy

Rodearte utilizes two primary typefaces from Google Fonts, managed via `next/font/google` to optimize performance and prevent Layout Shift (CLS).

Font	Variable	Role	Subsets
Noto Serif Display	<code>--font-serif</code>	Headings and brand accents	Latin
Montserrat	<code>--font-sans</code>	Body text and navigation	Latin

The fonts are injected into the DOM as CSS variables `src/app/layout.tsx:7-7`, `src/app/layout.tsx:13-13`. The `Montserrat` font is set as the default `font-sans` for the `<body>` element `src/app/layout.tsx:29-29`. Both fonts use `display: "swap"` to ensure text remains visible during font loading `src/app/layout.tsx:8-14`.

Implementation Diagram: Layout Structure

The following diagram illustrates how `RootLayout` wraps the application content and provides the CSS variable context for the design system.

Layout Hierarchy and CSS Variable Injection

```
graph TD
  subgraph "RootLayout [src/app/layout.tsx]"
    HTML["html (lang='es')"]
    BODY["body (.variable classes)"]
  end
```

```

    subgraph "Font Engine"
      NSD["Noto_Serif_Display"]
      MS["Montserrat"]
    end

    CHILDREN["{children}"]
  end

  NSD -- "injects --font-serif" --> BODY
  MS -- "injects --font-sans" --> BODY
  HTML --> BODY
  BODY --> CHILDREN
  CHILDREN --> HOME["Home [src/app/page.tsx]"]

```

Sources: `src/app/layout.tsx:1-34`

Page Composition

The Rodearte landing page is a single-page architecture composed within `src/app/page.tsx`. It assembles multiple specialized section components into a linear vertical flow.

Component Assembly

The `Home` component acts as the orchestrator, importing sections from `@/components/sections/*` and placing them within a `<main>` container that ensures a minimum screen height `src/app/page.tsx:10-35`.

The sequence of sections is as follows: 1. **Navbar**: Persistent navigation (placed outside `<main>` for global positioning) `src/app/page.tsx:13-13`. 2. **Hero**: The primary visual entry point `src/app/page.tsx:15-15`. 3. **Pilares**: Core brand values and philosophy `src/app/page.tsx:18-18`. 4. **QueEsRodearte**: Detailed description of the methodology `src/app/page.tsx:21-21`. 5. **Clases**: Service offerings and scheduling `src/app/page.tsx:24-24`. 6. **Mono**: Apparel and product showcase `src/app/page.tsx:27-27`. 7. **Pasaporte**: Engagement and attendance concept `src/app/page.tsx:30-30`. 8. **Footer**: Contact information and site map `src/app/page.tsx:32-32`.

Data Flow and Component Tree

The following diagram maps the relationship between the **Home** page and its constituent section components.

Rodearte Page Composition Tree

```
graph TD
    PAGE["Home (src/app/page.tsx)"]
    NAV["Navbar"]
    MAIN["main (.min-h-screen)"]
    FOOT["Footer"]

    subgraph "Sections"
        HERO["Hero"]
        PIL["Pilares"]
        QER["QueEsRodearte"]
        CLA["Clases"]
        MON["Mono"]
        PAS["Pasaporte"]
    end

    PAGE --> NAV
    PAGE --> MAIN
    PAGE --> FOOT

    MAIN --> HERO
    MAIN --> PIL
    MAIN --> QER
    MAIN --> CLA
    MAIN --> MON
    MAIN --> PAS
```

Sources: [src/app/page.tsx:1-35](#)

Key Layout Classes

- **min-h-screen** : Applied to the `<main>` tag to ensure the background and layout span at least the full viewport height even if content is sparse [src/app/page.tsx:14-14](#).
- **font-sans** : Applied at the `<body>` level to ensure **Montserrat** is the fallback font for all child components unless overridden [src/app/layout.tsx:29-29](#).

Sources: src/app/layout.tsx:1-34, src/app/page.tsx:1-35

Sheet (Drawer) Component

The Sheet component is a slide-out panel (often referred to as a "Drawer") used for mobile navigation, supplementary forms, and contextual information. It is built upon the `@radix-ui/react-dialog` primitive, ensuring accessibility (ARIA patterns, focus trapping) while providing custom styling and animations via Tailwind CSS and `class-variance-authority` (CVA).

Component Architecture

The Sheet implementation is composed of several sub-components that manage the portal, overlay, content, and accessibility labels. It utilizes a declarative pattern where the `Sheet` component acts as a state provider for its children.

Component Relationship Diagram

The following diagram illustrates the relationship between the exported components and the underlying Radix UI primitives.

Sheet Component Hierarchy

```
graph TD
    subgraph "Radix UI Primitives"
        RP_Root["Dialog.Root"]
        RP_Portal["Dialog.Portal"]
        RP_Overlay["Dialog.Overlay"]
        RP_Content["Dialog.Content"]
        RP_Trigger["Dialog.Trigger"]
        RP_Close["Dialog.Close"]
    end

    subgraph "Rodearte Sheet Components"
        S_Root["Sheet (Exported)"] --- RP_Root
        S_Trig["SheetTrigger"] --- RP_Trigger
        S_Port["SheetPortal"] --- RP_Portal
    end
```

```

    S_Over["SheetOverlay"] --- RP_Overlay
    S_Cont["SheetContent"] --- RP_Content
    S_Close["SheetClose"] --- RP_Close
  end

  S_Root --> S_Trig
  S_Root --> S_Port
  S_Port --> S_Over
  S_Port --> S_Cont
  S_Cont --> S_Close

```

Sources: `src/components/ui/sheet.tsx:10-16`, `src/components/ui/sheet.tsx:56-74`

Implementation Details

Side Variants and CVA

The component supports four entry directions (top, bottom, left, right) defined via the `sheetVariants` CVA configuration. This configuration handles both the positioning and the corresponding entry/exit animations.

Side	CSS Classes	Animation (In/Out)
top	<code>inset-x-0 top-0 border-b</code>	<code>slide-in-from-top</code> / <code>slide-out-to-top</code>
bottom	<code>inset-x-0 bottom-0 border-t</code>	<code>slide-in-from-bottom</code> / <code>slide-out-to-bottom</code>
left	<code>inset-y-0 left-0 h-full w-3/4 border-r</code>	<code>slide-in-from-left</code> / <code>slide-out-to-left</code>
right	<code>inset-y-0 right-0 h-full w-3/4 border-l</code>	<code>slide-in-from-right</code> / <code>slide-out-to-right</code>

Sources: `src/components/ui/sheet.tsx:33-50`

Sub-Component Definitions

- **SheetOverlay** : A full-screen backdrop that dims the background (`bg-black/80`). It uses Radix state attributes (`data-state`) to trigger fade animations `src/components/ui/sheet.tsx:18-30`.
- **SheetContent** : The main container for the drawer content. It automatically wraps its children in a `SheetPortal` and `SheetOverlay` , and includes a built-in `SheetClose` button (the "X" icon) in the top-right corner `src/components/ui/sheet.tsx:56-74`.
- **SheetHeader** / **SheetFooter** : Layout utilities for consistent spacing within the drawer. `SheetFooter` automatically switches from a vertical stack on mobile to a horizontal row on larger screens `src/components/ui/sheet.tsx:77-103`.
- **SheetTitle** / **SheetDescription** : Semantic components for accessibility. These map to `Dialog.Title` and `Dialog.Description` to ensure screen readers properly identify the drawer's purpose `src/components/ui/sheet.tsx:105-127`.

Data Flow and Animations

The Sheet component relies on Tailwind CSS's animation utilities and Radix UI's state management to handle transitions.

Animation Logic Flow

```
sequenceDiagram
    participant User
    participant "SheetTrigger" as ST
    participant "SheetContent" as SC
    participant "SheetOverlay" as SO

    User->>ST: Click Trigger
    ST->>SC: Set data-state="open"
    SC->>SC: Apply animate-in (duration-500)
    SC->>SC: Execute slide-in-from-[side]
    ST->>SO: Set data-state="open"
    SO->>SO: Apply fade-in-0

    User->>SC: Click Close Button
    SC->>SC: Set data-state="closed"
    SC->>SC: Apply animate-out (duration-300)
```

```
SC->>SC: Execute slide-out-to-[side]
S0->>S0: Apply fade-out-0
```

Sources: `src/components/ui/sheet.tsx:24-24`, `src/components/ui/sheet.tsx:34-44`, `src/components/ui/sheet.tsx:68-71`

Key Classes and Functions

Entity	Role	Source
<code>Sheet</code>	Alias for <code>SheetPrimitive.Root</code> , manages open/closed state.	<code>src/components/ui/sheet.tsx:10-10</code>
<code>SheetPortal</code>	Teleports the drawer to the end of the <code><body></code> to avoid z-index nesting issues.	<code>src/components/ui/sheet.tsx:16-16</code>
<code>SheetContent</code>	Functional component that applies <code>sheetVariants</code> based on the <code>side</code> prop.	<code>src/components/ui/sheet.tsx:56-74</code>
<code>cn(...)</code>	Utility used to merge default variant styles with custom <code>className</code> props.	<code>src/components/ui/sheet.tsx:64-64</code>
<code>X</code>	Icon from <code>lucide-react</code> used for the close button.	<code>src/components/ui/sheet.tsx:6-6</code>

Sources: `src/components/ui/sheet.tsx:1-140`

Static Assets

This section provides an overview of the static files served by the Rodearte application. These assets are located in the `public/` directory and include the high-resolution photography used for sections and product displays, as well as the comprehensive SVG logo library used for branding across different UI states.

Overview of the public/ Directory

The `public/` directory contains all non-code assets that are served directly to the browser. These assets are critical for the "Somatic Movement" aesthetic of the site, utilizing glassmorphism and high-quality imagery to convey the brand's identity.

Directory Structure

The assets are organized into two primary subdirectories:

Directory	Content Type	Primary Usage
<code>public/jpg/</code>	Photography & Backgrounds	Section backgrounds, product images, and placeholders.
<code>public/logos/</code>	SVG Branding	Responsive logos for Navbar, Footer, and scroll states.
<code>src/app/icon.svg</code>	Favicon / App Icon	Browser tab branding and Next.js metadata.

Mapping Assets to Code Entities

The following diagram illustrates how static assets are associated with specific React components within the `src/components/` directory.

Asset-to-Component Mapping

```
graph TD
  subgraph "public/jpg/"
    IMG_01["Rodearte_01-22.jpg"]
    IMG_AU["aboutUs.avif"]
    IMG_FB["fullbody.jpg"]
  end

  subgraph "public/logos/"
    L_1["1.svg (Gold)"]
    L_8["8.svg (White)"]
    L_6["6.svg (Dark)"]
  end

  subgraph "Code Entity Space (Components)"
```

```

    NAV["Navbar Component"]
    HERO["Hero Component"]
    QUES["QueEsRodearte Component"]
    MONO["Mono Component"]
    FOOT["Footer Component"]
  end

  L_1 --> NAV
  L_8 --> NAV
  L_6 --> FOOT
  IMG_01 --> MONO
  IMG_AU --> QUES
  IMG_FB --> HERO

```

Sources: src/app/layout.tsx:1-40, public/logos/1.svg:1-10, public/jpg/Rodearte_01-22.jpg:1-10

Photography Assets

The photography assets are the visual core of the Rodearte experience. They are primarily served using the Next.js `Image` component to ensure optimization, lazy loading, and appropriate sizing across devices.

- **Rodearte Series:** A collection of high-resolution images (e.g., `Rodearte_01-22.jpg`) used in the `Mono` section to showcase apparel.
- **Backgrounds:** Specific files like `aboutUs.avif` and `fullbody.jpg` serve as full-viewport or container-bounded backgrounds for narrative sections.
- **Placeholders:** The `-.png` file is used as a lightweight placeholder during image loading or for missing assets.

For a full catalog of images and their technical implementation details (sizes, priority, and fill patterns), see **Photography Assets**.

Sources: public/jpg/Rodearte_01-22.jpg:1-46

Logo SVG Library

Rodearte utilizes a library of 29 numbered SVG files to handle complex branding requirements. Because the site uses a scroll-responsive `Navbar` and a multi-themed `Footer`, the application swaps between different logo variants based on the UI state.

Key Branding Assets

Asset Path	Role	Usage Context
<code>public/logos/1.svg</code>	Primary Logo	Default Navbar state and Desktop Footer.
<code>public/logos/8.svg</code>	Light Variant	Navbar when scrolled (overlying dark content).
<code>public/logos/6.svg</code>	Mobile Variant	Compact Footer view for mobile devices.
<code>src/app/icon.svg</code>	System Icon	Generated favicon for the Next.js App Router.

Logo State Logic

The `Navbar` component dynamically selects the logo based on the `isScrolled` state provided by the `useScroll` hook.

Navbar Logo Logic

```
graph LR
  SCROLL["useScroll Hook"] -->|isScrolled: true| L8["logos/8.svg (White)"]
  SCROLL -->|isScrolled: false| L1["logos/1.svg (Gold)"]
  L8 --> UI["Navbar Display"]
  L1 --> UI
```

For details on the full library and the implementation of the app icon, see [Logo SVG Library](#).

Sources: `src/app/icon.svg:1-10`, `public/logos/1.svg:1-10`

UI Component Library

The Rodearte UI library is built upon **shadcn/ui** and **Radix UI**, providing a set of accessible, themeable, and reusable primitives. These components are located in `src/components/ui/` and utilize `class-variance-authority` (CVA) for managing style variants and `tailwind-merge` for class composition.

The library follows a "copy-and-paste" philosophy where components are owned by the project, allowing for granular customization of the design system's tokens and behaviors `components.json:1-20`.

Component Architecture

The following diagram illustrates how the UI library bridges the gap between Radix UI primitives and the Rodearte design system.

UI Primitive Composition

```
graph TD
    subgraph "Radix UI (Primitives)"
        R1["@radix-ui/react-slot"]
        R2["@radix-ui/react-dialog"]
    end

    subgraph "Rodearte UI Components (Code Entity Space)"
        B["Button (src/components/ui/button.tsx)"]
        S["Sheet (src/components/ui/sheet.tsx)"]
    end

    subgraph "Styling Utilities"
        CVA["cva (class-variance-authority)"]
        CN["cn (src/lib/utils.ts)"]
    end

    R1 --> B
    R1 --> S
    R2 --> S
    CVA --> B
    CVA --> S
    CN --> B
    CN --> S
```

Sources: `src/components/ui/button.tsx:1-8`, `src/components/ui/sheet.tsx:1-9`

Core Components

The library is categorized into foundational primitives used across all sections of the landing page.

Button Component

The `Button` component is the primary interaction element. It supports multiple visual variants (e.g., `default`, `outline`, `ghost`) and sizes defined via a `buttonVariants` CVA object `src/components/ui/button.tsx:7-34`. It uses the `Slot` primitive from Radix UI to support the `asChild` pattern, allowing the button styles to be applied to other elements like links or custom components `src/components/ui/button.tsx:44-46`.

For details, see [Button Component](#).

Sheet (Drawer) Component

The `Sheet` component provides an overlay panel (drawer) that slides in from the edges of the screen. It is built on top of the `@radix-ui/react-dialog` primitive `src/components/ui/sheet.tsx:4-10`. In the Rodearte project, it is primarily used for the mobile navigation menu, supporting different entry directions via the `side` variant (top, bottom, left, right) `src/components/ui/sheet.tsx:37-44`.

For details, see [Sheet \(Drawer\) Component](#).

Configuration and Utilities

The UI library is integrated into the project via `components.json`, which defines the path aliases and Tailwind CSS configuration `components.json:6-19`.

Component Mapping to Code Entities | Component | Primary File | Key

Dependencies			:---		:---		:---		Button		<code>src/components/ui/button.tsx</code>		<code>class-variance-authority</code>	,	<code>@radix-ui/react-slot</code>		Sheet		<code>src/components/ui/sheet.tsx</code>
--------------	--	--	------	--	------	--	------	--	---------------	--	---	--	---------------------------------------	---	-----------------------------------	--	--------------	--	--

| `@radix-ui/react-dialog` , `lucide-react` | | **Styling** | `src/lib/utils.ts` | `clsx` ,
`tailwind-merge` |

Library Structure Diagram

```
classDiagram
    class "buttonVariants" {
        +default
        +destructive
        +outline
        +secondary
        +ghost
        +link
    }
    class "sheetVariants" {
        +top
        +bottom
        +left
        +right
    }
    class "cn utility" {
        +inputs: ClassValue[]
    }

    "Button" ..> "buttonVariants" : uses
    "SheetContent" ..> "sheetVariants" : uses
    "Button" ..> "cn utility" : merges classes
    "Sheet" ..> "cn utility" : merges classes
```

Sources: `src/components/ui/button.tsx:7-34`, `src/components/ui/sheet.tsx:33-50`,
`components.json:13-19`

Utilities, Hooks, and Types

This section covers the foundational infrastructure of the Rodearte codebase, including the utility functions for styling, custom React hooks for browser interaction, and global TypeScript interfaces that define the data structures used across multiple components.

Class Merging Utility

The project utilizes a standard utility function, `cn`, to manage dynamic Tailwind CSS class names. This utility is essential for building flexible UI components that need to merge base styles with conditional classes or external overrides.

Implementation Details

The `cn` function is a wrapper around two industry-standard libraries: 1. `clsx`: A utility for constructing `className` strings conditionally `src/lib/utls.ts:1-1`. 2. `tailwind-merge`: Specifically designed to resolve Tailwind CSS class conflicts (e.g., ensuring `px-4` overrides `p-2`) `src/lib/utls.ts:2-2`.

The function accepts a spread of `ClassValue` inputs and returns a single merged string `src/lib/utls.ts:4-6`.

Data Flow: Styling

Step	Process	Code Entity
1	Input raw classes and conditions	<code>cn("base-style", isActive && "active-style", props.className)</code>
2	Conditional logic resolution	<code>clsx</code> <code>src/lib/utls.ts:5-5</code>
3	Conflict resolution	<code>twMerge</code> <code>src/lib/utls.ts:5-5</code>
4	Output	Optimized string for the <code>className</code> attribute

Sources: - `src/lib/utls.ts:1-8`

Custom Hooks: useScroll

The `useScroll` hook is a client-side utility used to track the user's vertical scroll position relative to a specific threshold. In the Rodearte application, this is primarily used by the `Navbar` to trigger visual transitions (such as logo swaps or background opacity changes) when the user moves away from the top of the page.

Technical Specification

The hook encapsulates the following logic: - **State Management:** Maintains an `isScrolled` boolean state `src/hooks/useScroll.ts:6-6`. - **Event Handling:** Attaches a passive scroll listener to the `window` object `src/hooks/useScroll.ts:17-17`. -

Threshold Logic: Compares `window.scrollY` against a configurable `threshold` (defaulting to 50 pixels) `src/hooks/useScroll.ts:5-11`. - **Lifecycle:** Includes a cleanup function to remove the event listener when the component unmounts, preventing memory leaks `src/hooks/useScroll.ts:19-21`.

Scroll Detection Logic

The diagram below illustrates the lifecycle and state transitions within the `useScroll` hook.

Logic Flow: useScroll Hook

```
graph TD
    subgraph "Hook Lifecycle"
        A["Mount"] --> B["handleScroll() Initial Check"]
        B --> C["Add window 'scroll' Event Listener"]
        C --> D["Scroll Event Triggered?"]
        D -- "Yes" --> E["Compare window.scrollY > threshold"]
        E -- "True" --> F["setIsScrolled(true)"]
        E -- "False" --> G["setIsScrolled(false)"]
        D -- "Unmount" --> H["Remove Event Listener"]
    end

    subgraph "External Consumer (e.g., Navbar)"
        F --> I["Apply 'scrolled' Styles"]
        G --> J["Apply 'transparent' Styles"]
    end
```

Sources: - `src/hooks/useScroll.ts:1-27`

Global Type Definitions

The application defines shared TypeScript interfaces to ensure type safety across navigation and feature-based components. These are centralized to provide a single source of truth for the data shapes used in the landing page sections.

NavItem Interface

Used for defining navigation links in both the desktop `Navbar` and the mobile `Sheet` menu. - `label` : The display text for the link `src/types/index.ts:4-4`. - `href` : The destination anchor (e.g., `#classes`) or URL `src/types/index.ts:5-5`.

Feature Interface

Used to model informational blocks, such as the pillars of the brand or service descriptions. - `title` : The heading of the feature `src/types/index.ts:9-9`. - `description` : Detailed text content `src/types/index.ts:10-10`. - `icon` : An optional string identifier for an icon or asset path `src/types/index.ts:11-11`.

Data Structure Mapping

The following diagram bridges the natural language concepts of the website's content to the code entities defined in the types file.

Conceptual Mapping: Rodearte Data Types

```
classDiagram
    class "Navigation Menu" as NavMenu {
        +String label
        +String href
    }
    class "Brand Pillar / Service" as BrandFeature {
        +String title
        +String description
        +String icon (optional)
    }

    class NavItem {
        <<interface>>
        +label: string
        +href: string
    }
    class Feature {
        <<interface>>
        +title: string
        +description: string
        +icon?: string
    }
```

```
NavMenu ..> NavItem : "Implemented as"  
BrandFeature ..> Feature : "Implemented as"
```

Sources: - [src/types/index.ts:1-14](#)
